Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO2 – Assignment 2: Brute-Force and Exhaustive-Search Techniques – Predict, Design, Implement, Measure, Analyse, Evaluate, Diagnose, Redesign and Validate
Course Outcome Assessed	Apply brute-force and exhaustive-search techniques (Selection, Bubble and Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search) and justify design decisions with operation-count evidence and mathematical reasoning.
Student Reg. No.	192473003
Student Name	Saran. S

DAA ASSIGNMENT 2 – CO2 REPORT

Scenario – Brute-Force and Exhaustive-Search Study of Six Engineering Problems

Six real-world engineering problems are studied in this report: order sorting in an e-commerce fulfilment centre (Q1), tool tracking in an aircraft maintenance facility (Q2), signature screening in security logs (Q3), collision-risk monitoring for delivery drones (Q4), boundary monitoring of a wildlife reserve (Q5) and payload planning for a humanitarian airlift (Q6). For each problem the report follows the same chain: **predict** the behaviour, **design** the test data, **implement** the algorithm, **measure** operation counts, **analyse** the result and **evaluate** a design decision. Q7 then selects one solution, **diagnoses** its bottleneck, **redesigns** it and **validates** the redesign against the original.

Given information:

- Nature of assignment: Individual; duration: 1 week; total marks: 50 (Q1 = 6, Q2 = 4, Q3 = 6, Q4 = 6, Q5 = 7, Q6 = 8, Q7 = 13)
- Experimental environment: Python 3.12.3 on a single x86-64 CPU core, standard library only (matplotlib was used only for the figures)
- Primary evidence: operation counts (comparisons, swaps/moves, character comparisons, distance calculations, side tests, candidates examined). Execution time is reported as additional evidence only and is the best of several runs measured with time.perf_counter()
- Reproducibility: every random data set uses a fixed seed, so every table in this report can be regenerated exactly by the source code in Appendix A
- Unit of cost: one *basic operation* is the step executed most often in each algorithm, defined separately for each question

(a) Understand the Assignment

The assignment does not ask for textbook algorithms alone. It asks for evidence: what will happen, why it will happen, what did happen, and what should be done about it. The goal of this section is to fix the problem, the input size, the basic operation and the expected growth of each question before any experiment is run.

Question	Input size	Technique and basic operation counted	Expected growth (theory)	Marks
Q1 – Order sorting	n orders	Selection, Bubble, Insertion sort; key comparisons and swaps/moves	$\Theta(n^2)$ worst; Insertion $\Theta(n)$ on sorted input	6
Q2 – Tool tracking	n active records	Sequential search; Tool-ID comparisons	$\Theta(n)$ worst, $(n+1)/2$ average	4
Q3 – Log screening	text length n, pattern length m	Brute-force string matching; character comparisons	$\Theta(nm)$ worst, $\Theta(n)$ best	6
Q4 – Drone monitoring	n drones	Brute-force closest pair; distance calculations	$n(n-1)/2 = \Theta(n^2)$	6
Q5 – Reserve boundary	n stations	Brute-force convex hull; side tests (cross products)	$\Theta(n^3)$ worst case	7
Q6 – Airlift planning	n packages	Exhaustive search over subsets; candidates examined	2^n candidates	8
Q7 – Design challenge	As Q4	Diagnose, redesign (pruned sweep), validate	Target: far below $n(n-1)/2$	13

Interpretation: five of the six brute-force solutions are polynomial ($\Theta(n)$ to $\Theta(n^3)$) and Q6 is exponential. The assignment therefore covers the whole range from algorithms that scale comfortably to one that becomes unusable after a few dozen inputs. The central question in every part is the same: *how much work does the algorithm really do on this particular input, and is that acceptable for the stated operating conditions?*

Notation and assumptions

Notation	Meaning	Used in this report for
n	Number of items processed (orders, records, drones, stations, packages) or text length in Q3	Input size in every question
m	Pattern length (Q3)	String-matching cost $\Theta(nm)$
C(n, 2)	Number of ways to choose 2 items from n, equal to $n(n-1)/2$	Pairs in Q4 and Q5
inv(A)	Number of inversions: pairs $i < j$ with $A[i] > A[j]$	Explaining swaps/moves in Q1
$f(n) = \Theta(g(n))$	f is bounded above and below by constant multiples of g for large n	Growth statements

The following assumptions are used throughout the report:

- Every counted basic operation is treated as one unit of cost. Counts are exact: they are produced by counters inside the code, not estimated.
- Execution times are measured on one core of the machine used for the experiments. They depend on the interpreter and hardware, so they are used only to support, never to replace, the operation counts.
- Where a question requires a business assumption (for example the lookup rate in Q2 or the response-time limit in Q4 and Q6), the assumption is stated explicitly and justified at that point. These are modelling choices made by me, not data supplied by the assignment.
- Random test data are produced with fixed seeds. Where a result could depend on the seed (for example the random-data rows), the formula-based prediction is also given so that the result can be judged against theory.
- The reference implementations used to cross-check answers (dynamic programming in Q6, the monotone chain in Q5, full brute force in Q7) are used only for verification and never inside the algorithm being evaluated.

How the growth rates compare

The table below shows how far apart the growth rates studied in this assignment really are. The values are computed directly from the formulas.

n	$n \log_2 n$	$n^2/2$	$n^3/2$	2^n
10	33	50	500	1,024
20	86	200	4,000	1,048,576
30	147	450	13,500	1,073,741,824
40	213	800	32,000	1.10×10^{12}
50	282	1,250	62,500	1.13×10^{15}
100	664	5,000	500,000	1.27×10^{30}
1,000	9,966	500,000	500,000,000	1.07×10^{301}

At n = 1,000 the cubic and quadratic algorithms need about 5×10^8 and 5×10^5 operations, which a computer handles easily. At n = 50 the exhaustive-search count 2^{50} is already about 10^{15}. This difference is the reason the same word, “brute force”, can describe a perfectly acceptable solution in one question and an unusable one in another.

Method used for every question

Each question follows the same six steps, mirroring the chain given in the assignment:

- **Predict** the behaviour from the algorithm’s structure and write the prediction down together with its algorithmic reason, before any program is run.
- **Design** deliberately contrasting inputs that expose the differences the prediction relies on.
- **Implement** the algorithm with built-in counters for the primary measure.
- **Measure** counts for several input sizes and record the raw numbers (Appendix B).
- **Analyse** by comparing prediction with observation and explaining every difference.
- **Evaluate** the engineering decision that the evidence supports, including its limits.

Q1 – Sorting Under Different Data Conditions (6 Marks)

An e-commerce fulfilment centre sorts customer orders by a dispatch priority score. The development team is choosing between Selection Sort, Bubble Sort and Insertion Sort, but the incoming data may be random, nearly sorted, already sorted or badly disordered. The question is whether one algorithm should always be preferred. The report answers it by predicting, testing and measuring each algorithm on deliberately chosen data.

► Step 1: Algorithms and the operations that are counted

Algorithm	Idea	Counted operations	Stable?
Selection Sort	In pass i find the smallest element of the unsorted part and swap it into position i.	Comparisons: each test A[j] < A[min]. Swaps: exchanges actually performed (one per pass at most).	No (long-distance swap)
Bubble Sort (with early-exit flag)	Repeatedly compare adjacent elements and swap them if out of order; stop after a pass with no swap.	Comparisons: each test A[j] > A[j+1]. Swaps: adjacent exchanges.	Yes
Insertion Sort	Insert A[i] into the already sorted prefix by shifting larger elements one place right.	Comparisons: each test A[j] > key. Moves: elements shifted.	Yes

The same test data and the same counters are used for all three algorithms, so the counts are directly comparable. The early-exit flag is included in Bubble Sort because it is the usual textbook improvement and gives Bubble Sort its best chance on sorted data.

► Step 2: Prediction before implementation

The predictions below were written from the structure of each algorithm, before any program was run. Let n be the number of orders and let inv(A) be the number of inversions in the input, that is, the number of pairs that are in the wrong order.

Algorithm	Already sorted	Reversed	All keys equal	Nearly sorted (95% in order)
Selection	n(n−1)/2 comparisons, 0 swaps	n(n−1)/2 comparisons, about n/2 swaps	n(n−1)/2 comparisons, 0 swaps	n(n−1)/2 comparisons, at most n−1 swaps
Bubble	n−1 comparisons (one pass), 0 swaps	n(n−1)/2 comparisons and swaps	n−1 comparisons, 0 swaps	Still close to n(n−1)/2 comparisons; swaps = inv(A)
Insertion	n−1 comparisons, 0 moves	About n(n−1)/2 comparisons and moves	n−1 comparisons, 0 moves	About n + inv(A) comparisons; moves = inv(A)

Algorithmic reasons.

- *Selection Sort ignores the data.* Its inner loop always scans the whole unsorted part to find the minimum, and nothing in the loop can stop early. The comparison count is therefore the fixed sum (n−1) + (n−2) + … + 1 = n(n−1)/2 for every input. Only the number of swaps varies, and it can never exceed n−1 because each pass makes at most one swap.

- *Insertion Sort pays only for disorder.* The inner loop shifts an element left past every larger element and stops at the first smaller one. Each shift removes exactly one inversion, so the number of moves equals inv(A), and the number of comparisons is about inv(A) + n − 1. Sorted input has inv(A) = 0, so the cost is linear.
- *Bubble Sort pays for disorder and for distance.* Every swap also removes exactly one inversion, so swaps = inv(A). However, a small element can move left by only one place per pass. If a small element sits near the end of the array, about n passes are needed and each pass still scans the unsorted part, so the early-exit flag does not help. I therefore predict that Bubble Sort will stay close to n(n−1)/2 comparisons whenever even one small value is far from its place.

$$\text{swaps (Bubble)} = \text{moves (Insertion)} = \text{inv}(A)$$

For randomly ordered keys every pair is inverted with probability ½, so the expected number of inversions is n(n−1)/4. For a nearly sorted list made of a sorted block of n−k orders followed by k new orders with random priorities, each new order has on average (n−k)/2 larger orders in front of it, so I predict inv(A) ≈ k(n−k)/2.

► Step 3: Design of the test data

Two deliberately contrasting data sets and one edge case are required. Four supporting data sets were added to connect the experiment to the fulfilment-centre situation. All sets contain n = 1,000 priority scores (and n = 2,000 for the scaling check); the random values are integers from 1 to 10n, generated with a fixed seed.

Label	Construction	Why it is included
A – Sorted (contrast 1)	Random scores sorted ascending	Best case for adaptive algorithms; shows which algorithms notice that no work is needed
B – Reversed (contrast 2)	Sorted scores in descending order	Worst case: every pair is inverted, so inv(A) = n(n−1)/2
C – All equal (edge case)	Every score is 5	No inversions at all and many ties; tests whether strict comparisons and the early-exit flag behave as predicted
D – Random	Uniform random scores	Average case, inv(A) ≈ n(n−1)/4
E – Nearly sorted, appended	950 sorted scores followed by 50 new random scores (5%)	Models the fulfilment centre: old orders in order, a new batch at the end
F – Nearly sorted, scattered	950 sorted scores with 50 random scores inserted at random positions	Same 95% order, but the disorder is spread through the list
G – Three distinct values	Each score drawn from {1, 2, 3}	Heavy duplication; checks the effect of ties on counts

► Step 4: Implementation

The three algorithms are implemented in Python with counters (full source in Appendix A). Pseudocode of the counted loops:

```
SelectionSort(A[0..n-1])
FOR i <- 0 TO n-2 DO
```

```
BubbleSort(A[0..n-1])
FOR pass <- 0 TO n-2 DO
```

```
  min <- i                                    swapped <- false
  FOR j <- i+1 TO n-1 DO                      FOR j <- 0 TO n-2-pass DO
    comparisons++                               comparisons++
    IF A[j] < A[min] THEN min <- j              IF A[j] > A[j+1] THEN
  END FOR                                         swap(A[j], A[j+1]); swaps++
  IF min != i THEN                                swapped <- true
    swap(A[i], A[min]); swaps++                 END FOR
END FOR                                         IF NOT swapped THEN STOP
                                              END FOR

InsertionSort(A[0..n-1])
FOR i <- 1 TO n-1 DO
  key <- A[i];  j <- i-1
  WHILE j >= 0 DO
    comparisons++
    IF A[j] > key THEN A[j+1] <- A[j]; moves++; j <- j-1
    ELSE BREAK
  END WHILE
  A[j+1] <- key
END FOR
```

Every output was checked against Python's sorted() before its counts were accepted, so no count in this report comes from an incorrect sort.

► Step 5: Observed results

Table 1 gives the exact counts for n = 1,000. For Selection and Bubble Sort the second number of each pair is swaps; for Insertion Sort it is element moves.

Data set	Selection comps	Sel. swaps	Bubble comps	Bub. swaps	Insertion comps	Ins. moves
A – Sorted	499,500	0	999	0	999	0
B – Reversed	499,500	522	499,500	499,455	499,497	499,455
C – All equal	499,500	0	999	0	999	0
D – Random	499,500	997	499,200	258,340	259,331	258,340
E – Near-sorted (appended)	499,500	987	498,972	24,010	25,009	24,010
F – Near-sorted (scattered)	499,500	848	491,874	14,767	15,765	14,767
G – 3 distinct values	499,500	672	437,372	157,632	158,631	157,632

Table 1: Exact operation counts, n = 1,000.

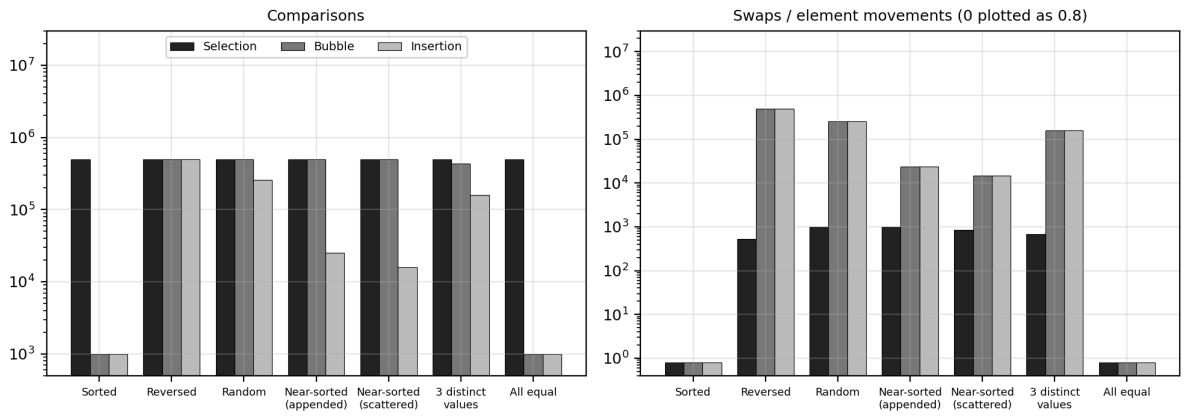


Figure 1: Comparisons (left) and swaps/movements (right) for n = 1,000 (logarithmic vertical axes).

Data set	Selection (ms)	Bubble (ms)	Insertion (ms)
A – Sorted	22.06	0.06	0.13
B – Reversed	21.25	76.42	38.35
C – All equal	19.81	0.04	0.07
D – Random	20.59	44.19	20.81
E – Near-sorted (appended)	20.10	24.48	2.09
F – Near-sorted (scattered)	20.36	24.63	1.33
G – 3 distinct values	20.12	34.04	13.37

Table 2: Execution time (best of 3 runs), n = 1,000. Shown as supporting evidence only.

Data set (n = 2,000)	Selection comps	Bubble comps	Insertion comps	Insertion ÷ Selection
A – Sorted	1,999,000	1,999	1,999	0.0010
B – Reversed	1,999,000	1,999,000	1,998,998	1.0000
C – All equal	1,999,000	1,999	1,999	0.0010
D – Random	1,999,000	1,997,965	1,010,785	0.5056
E – Near-sorted (appended)	1,999,000	1,997,569	94,729	0.0474
F – Near-sorted (scattered)	1,999,000	1,984,804	61,606	0.0308
G – 3 distinct values	1,999,000	1,763,359	663,584	0.3320

Table 3: Scaling check at n = 2,000 (doubling n).

Prediction	Observed (n = 1,000)	Verdict
Selection: n(n−1)/2 = 499,500 comparisons for every data set	499,500 in all seven data sets	Confirmed exactly
Selection: at most n−1 = 999 swaps	Maximum 997 swaps (random data); 0 for sorted and all-equal data	Confirmed
Bubble and Insertion on sorted data: n−1 = 999 comparisons	999 and 999; also 999 for all-equal data	Confirmed exactly
Bubble swaps = Insertion moves = inv(A)	Reversed: 499,455 = 499,455; random: 258,340 = 258,340	Confirmed in every data set
Reversed data: n(n−1)/2 = 499,500 swaps for Bubble	499,455 (the 45 missing swaps are equal-valued neighbours that are not inversions)	Confirmed; difference explained by duplicate scores
Random data: inv(A) ≈ n(n−1)/4 = 249,750	258,340 (3.4% above the mean; within normal random variation)	Confirmed
Near-sorted (appended): moves ≈ k(n−k)/2 = 23,750	24,010 moves	Confirmed (+1.1%)
Bubble on near-sorted data stays close to n(n−1)/2	498,972 comparisons (99.9% of n(n−1)/2); scattered: 491,874 (98.5%)	Confirmed – the early-exit flag almost never fires

Table 4: Prediction compared with observation.

► Step 6: Analysis of the observed behaviour

- **Selection Sort is insensitive to the data.** Its comparison count is identical (499,500) for all seven data sets and doubling n multiplies it by 4.00 (1,999,000 ÷ 499,500). Its single advantage is the small number of swaps, never more than n−1. It therefore has a real use only where writing to memory is far more expensive than reading, for example when records are written to flash storage.
- **Insertion Sort is the only algorithm whose cost follows the disorder.** It needs 999 comparisons on sorted and all-equal data, about 25,000 on the 95%-sorted data and about 259,000 on random data. Its cost is essentially n + inv(A), which is exactly what the prediction states.
- **Bubble Sort looks adaptive only on perfectly sorted data.** The early-exit flag rescues it on sorted and all-equal input (999 comparisons), but a handful of misplaced small values is enough to make it perform almost the full n(n−1)/2 comparisons. This is the most important surprise in the experiment: on the 95%-sorted data Bubble Sort does about 20 times more comparisons than Insertion Sort even though the two algorithms perform almost the same number of swaps or moves (24,010 for both). The difference comes from comparisons, not from data movement: Bubble Sort must keep scanning the sorted part of the list in every pass, while Insertion Sort stops as soon as an element is in place.
- **The all-equal edge case confirms the use of strict comparisons.** Because the algorithms test “greater than” and not “greater than or equal to”, equal keys are never swapped or shifted. Bubble Sort and Insertion Sort therefore do no work at all beyond one verifying pass, and the sort remains stable, which matters for orders with identical priority scores.
- **Duplicates reduce disorder.** With only three distinct values (data set G) the inversion count falls to 157,632, about 61% of the random-data value, because equal pairs are not inversions. The comparison counts of Bubble and Insertion Sort fall accordingly, while Selection Sort is unchanged.

► Step 7: Should one algorithm always be preferred?

No single algorithm wins everywhere, but the evidence is not evenly balanced either. Insertion Sort was never worse than Bubble Sort on any data set in either comparisons or movements, so Bubble Sort has no data condition in which it is the better engineering choice among these three. Selection Sort wins only when the number of writes is the scarce resource. The table summarises which algorithm the evidence favours under each condition.

Input condition	Best choice	Reason supported by the counts
Already sorted / all equal	Insertion (or Bubble with flag)	999 comparisons, no moves; Selection still performs 499,500 comparisons
Nearly sorted	Insertion	About 20 times fewer comparisons than the other two at n = 1,000
Random	Insertion	About half the comparisons of the others; same moves as Bubble
Reversed	Selection if writes are costly, otherwise a tie	Selection makes about 500 swaps, the others about 499,500; all make about $n^2/2$ comparisons

Input condition	Best choice	Reason supported by the counts
Writes are very expensive	Selection	At most n−1 swaps whatever the data

► Design decision: 95% of orders already in priority order

The fulfilment centre reports that about 95% of the orders are already in the correct priority order, with small batches of new orders added periodically. This is exactly the situation of data sets E and F, so the measured values apply directly.

n	Selection comps	Bubble comps	Insertion comps	Insertion moves	Bubble ÷ Insertion	Selection ÷ Insertion
1,000	499,500	498,972	25,009	24,010	20.0	20.0
2,000	1,999,000	1,997,569	94,729	92,730	21.1	21.1

Recommendation: Insertion Sort. The decision is defended with four points.

- **Cost follows the size of the new batch.** The work is about $n + k(n-k)/2$ comparisons for a batch of k new orders, so a batch of 5% costs roughly $5\% \times n^2/2$ instead of $n^2/2$. At n = 1,000 the measured counts are 25,009 (Insertion) against 498,972 (Bubble) and 499,500 (Selection), a factor of about 20. The factor stays near 20 at n = 2,000 (94,729 against 1,997,569 and 1,999,000).
- **It is stable.** Orders with equal priority keep their arrival order, which is a fairness requirement in a fulfilment centre. Selection Sort, as implemented here with long-distance swaps, does not guarantee this.
- **It is in place and simple.** It needs only one temporary variable, has no passes to wait for, and can sort the list as orders arrive.
- **Its advantage is evidence-based, not assumed.** It was the cheapest (or tied for cheapest) algorithm in comparisons on every data set that is not reversed, and never worse in movements than Bubble Sort.

Limits of the recommendation. The advantage depends on the 95% figure staying true. If the batches grow until the list is random, Insertion Sort costs about $n^2/4$ comparisons, still the lowest of the three but no longer cheap, and the centre should then change strategy. For this workload a better design than any of the three is to sort the small batch of new orders separately and merge it with the existing sorted list in a single linear pass, costing about $n + k \log k$ operations. That design is outside the scope of the three algorithms compared here, but the measured counts show where it would pay off: the 25,009 comparisons that Insertion Sort needs at n = 1,000 are already far above the roughly $1{,}000 + 50 \times 6 = 1{,}300$ that a merge would need.

Interpretation: the experiment answers the question posed at the start. The best algorithm depends on the input, but for the fulfilment centre’s data the dependence is strong enough to settle the choice: Insertion Sort. Predicting the behaviour from the inversion count was accurate to within about one per cent for the 95%-sorted data and exact for the sorted and all-equal data.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? (4 Marks)

An aircraft maintenance facility keeps a temporary list of specialised tools currently issued to technicians. Each record holds Tool ID, Technician ID, Issue Time and Work Bay. When a technician asks “Who currently has Tool T-184?” the system scans the list from the beginning until it finds the Tool ID. The list changes continuously as tools are issued and returned. This question investigates how many comparisons the scan needs, and under what conditions that is acceptable.

► Step 1: Cost model and prediction

Let n be the number of records in the list. The basic operation is the comparison of the requested Tool ID with the Tool ID of the current record. If the wanted record is at position p (counting from 1), the scan stops after exactly p comparisons. If the Tool ID is not in the list, every record is examined. The predictions are:

Search type	Comparisons predicted	Reason
Successful, first position	1	Found by the first comparison
Successful, position p	p	One comparison per record up to and including the match
Successful, average over all positions	(n + 1)/2	(1 + 2 + … + n)/n
Successful, last position	n	All records examined
Unsuccessful	n	The scan can only stop at the end of the list, because the list is not sorted

$$\frac{1+2+\cdots+n}{n} = \frac{n+1}{2}$$

The unsuccessful search is the most expensive and cannot be shortened, because with an unsorted list the only proof that a tool is absent is to inspect every record. This matters in practice: a technician who mistypes a Tool ID, or asks about a tool that has been returned, triggers the worst case.

► Step 2: Experimental design

The experiment was designed to cover the three conditions in the question.

- **Successful searches at different positions:** the wanted record placed first, at 25%, 50% and 75% of the list, and last. The average over all positions is also measured (every position for n ≤ 1,000, every tenth or hundredth position for the larger sizes).
- **Unsuccessful searches:** the identifier “T-none” that does not occur in any list.
- **Different list sizes:** n = 10, 100, 1,000, 10,000 and 100,000 records, each record being (Tool ID, Technician ID, Issue Time, Work Bay) with unique, randomly generated Tool IDs (fixed seed).
- **Reference for comparison:** a hash index (dictionary) keyed by Tool ID, to see what an indexed lookup costs for the same lists.

- **Dynamic workload:** a simulation of 4,000 operations on a list that keeps a steady size, with 50% “who has it?” lookups (80% for tools that exist, 20% for tools that do not) and 25% returns (search, then removal and re-issue). This reflects that the list is continuously changing, and that a return also needs a search.

► Step 3: Observed comparison counts

List size n	First	25%	50%	75%	Last	Average (successful)	Unsuccessful	Hash index
10	1	3	6	8	10	5.5	10	1 probe
100	1	26	51	76	100	50.5	100	1 probe
1,000	1	251	501	751	1,000	500.5	1,000	1 probe
10,000	1	2,501	5,001	7,501	10,000	4,996.0	10,000	1 probe
100,000	1	25,001	50,001	75,001	100,000	49,951.0	100,000	1 probe

Table 5: Comparisons needed by sequential search (exact counts).

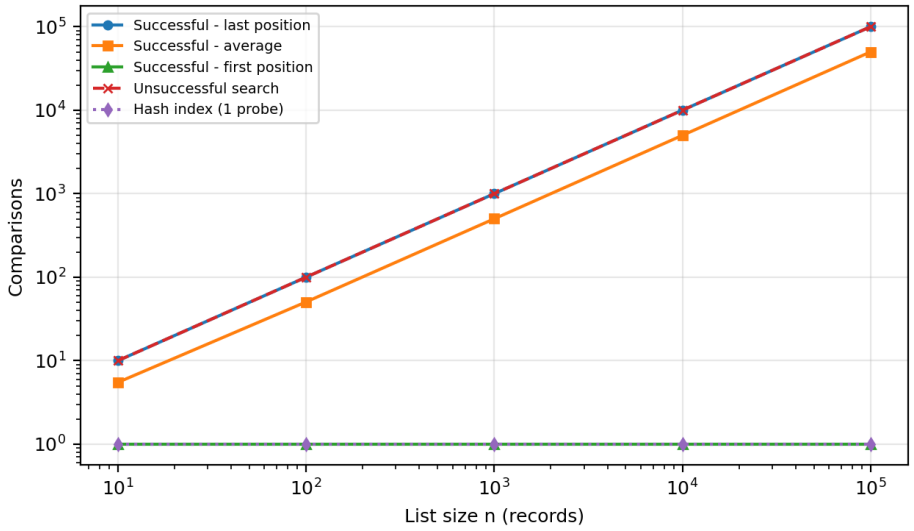


Figure 2: Comparisons against list size (both axes logarithmic). Sequential search grows as a straight line of slope 1, the hash index stays flat.

► Step 4: Analysis of successful and unsuccessful searches

- **Position determines cost exactly.** The counts are exactly 1 for the first position and n for the last, and exactly p + 1 for the record stored at index p. The 25%, 50% and 75% columns confirm the linear relation: 25,001, 50,001 and 75,001 comparisons for n = 100,000.
- **The average is about n/2.** The measured average is 5.5, 50.5 and 500.5 for n = 10, 100 and 1,000, matching (n + 1)/2 exactly. For n = 10,000 and 100,000 the average was taken over every 10th and every 100th position, which gives 4,996.0 and 49,951.0, within 0.1% of the exact values 5,000.5 and 50,000.5.
- **Unsuccessful searches cost the same as the worst successful search**, n comparisons, and twice the successful average. In the dynamic simulation the mixed lookups (20% unsuccessful) averaged 296.7 comparisons for N = 500 and 6,017.7 for N = 10,000, in agreement with the predicted $0.8 \times (N+1)/2 + 0.2 \times N \approx 0.6N$ (300 and 6,000).

- **Size scaling is linear.** Multiplying n by 10 multiplies every count by 10. The simulation shows the same: raising the list from 500 to 10,000 records (×20) raised the average return search from 253.3 to 5,104.5 comparisons (×20.2).
- **Measured cost per comparison.** Timing the unsuccessful scan and dividing by n gives about 43 ns per comparison in this environment (the figures for the five list sizes lie between 36 and 57 ns). The hash lookup took about 30 ns for every list size, so for the list of 100,000 records an indexed lookup used one probe where the scan used up to 100,000 comparisons (about 4.4 ms against 0.03 µs in time).

► Step 5: Operating boundary for one hangar

Whether 50 or 5,000 comparisons are acceptable depends on the number of records *and* on how often the search is requested. Two acceptability criteria were chosen, with their justification:

- **Latency criterion:** the worst-case single lookup must finish within 10 ms, so that a technician at a terminal perceives the answer as immediate.
- **Load criterion:** the total search work must stay below 10% of one CPU core, so that issuing, returning, the user interface and logging keep enough capacity.

Using the measured cost c ≈ 43 ns per comparison, the latency criterion allows n ≤ 0.01 / c ≈ 234,742 records, far more than a hangar holds. The load criterion depends on the request rate q (lookups and returns per second). With the mix measured above, the average lookup needs about 0.6n comparisons, so the load is 0.6 · q · n · c and the criterion becomes:

$$0.6qnc \le 0.10 \iff n \cdot q \le \frac{0.10}{0.6c}$$

The boundary is therefore a limit on the **product** n · q. With c = 43 ns the limit is n · q ≈ 3,912,363 comparisons-worth of records per second. For example, 500 records can be searched about 7,825 times per second, whereas 100,000 records can be searched only about 39 times per second before the 10% load is reached.

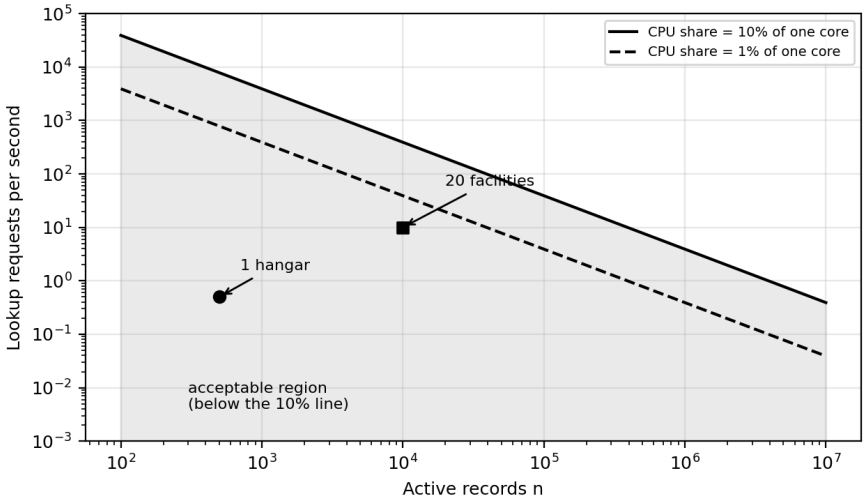


Figure 3: Operating boundary. The solid line is the 10% CPU limit; points below it are acceptable. The two plotted points are the assumed workloads of Step 6.

Recommendation for a single hangar. I would use sequential search for the single-hangar system. The list is short (a few hundred active records), it is changing constantly so there is no sorted structure

to maintain, and an average lookup needs a few hundred comparisons, a few microseconds. Adding an index would add code, memory and an update step on every issue and return for no measurable benefit. Sequential search is a reasonable engineering choice when n is small, q is low, the data are unsorted and frequently changing, and each search is needed only occasionally.

When I would stop recommending it: when $n \cdot q$ approaches the boundary above, when the list must serve many simultaneous users, when response-time guarantees must hold under peak load, or when the cost per comparison is much higher than measured here (for example a database row fetched over a network).

► Step 6: Changed condition – 20 maintenance facilities

The organisation now operates 20 facilities, with substantially more active records and more lookup requests. To re-evaluate the recommendation, the following workload is **assumed** (these are my modelling assumptions, not data supplied by the question): one hangar holds 500 active records and receives 0.5 requests per second (about 1,800 per hour, counting lookups and returns); a central system for 20 facilities holds $20 \times 500 = 10{,}000$ records and receives $20 \times 0.5 = 10$ requests per second.

Quantity	One hangar	20 facilities (one central list)	Change
Active records n	500	10,000	× 20
Requests per second q	0.5	10	× 20
Average comparisons per request (simulation)	296.7 (lookup), 253.3 (return)	6,017.7 (lookup), 5,104.5 (return)	× 20
Comparisons per second ($0.6 \cdot n \cdot q$)	150	60,000	× 400
CPU load at c = 43 ns	0.00064%	0.256%	× 400
Product $n \cdot q$	250	100,000	× 400

The key effect is multiplicative. Both the list length and the number of requests grow by 20, so the total work grows by $20 \times 20 = 400$. Under the measured speed the central system still lies inside the boundary (0.26% load against the 10% limit, a margin of nearly 40 times). The re-evaluation therefore depends strongly on the cost per comparison, which is machine- and system-dependent. The table below shows the boundary for three assumed costs.

Cost per comparison c	Boundary $n \cdot q$ (10% load)	Hangar ($n \cdot q$ = 250)	20 facilities ($n \cdot q$ = 100,000)
43 ns (measured)	3,912,363	within limit	within limit (0.3% load)
500 ns (assumed: object records, string compare)	333,333	within limit	within limit (3.0% load)
2 µs (assumed: remote database row)	83,333	within limit	exceeds limit (12% load)

Re-evaluated recommendation. For the 20-facility system I would no longer recommend a single sequential list. The reasons come from the evidence: the work has increased 400-fold; the safety margin falls from several orders of magnitude to a small multiple under pessimistic but realistic costs; requests will arrive in bursts (for example at shift changes) far above the average rate; and every return needs a search too, so the load is higher than the lookup rate alone. A hash index keyed by Tool ID

costs one probe per request whatever n is, can be updated in constant time on every issue and return, and needs only a modest amount of memory for 10,000 records. If a hash table is not available, keeping one list per facility restores n = 500, because most questions concern a tool in the technician’s own facility, but a search across all 20 lists then costs the same as one central scan.

Interpretation: sequential search is an appropriate engineering choice in the original situation, not because it is efficient, but because $n \cdot q$ is tiny and the data are unsorted and changing. The same algorithm becomes a poor choice not when it fails, but when the product $n \cdot q$ grows 400-fold and the safety margin disappears. The decision rule is therefore the boundary $n \cdot q \le 0.10 / (0.6c)$, reassessed whenever either factor changes.

Q3 – Engineer the Behaviour of Brute-Force String Matching (6 Marks)

A security operations centre searches large text logs for known event signatures, which are character patterns. The initial system uses brute-force string matching: the pattern is placed at the first position of the text, compared left to right, and if a mismatch occurs the pattern is shifted one position and the comparison restarts from the pattern's first character. This question investigates how the *structure* of the text and pattern, and not only their length, determines the work done.

► Step 1: Algorithm and basic operation

Let the text have n characters and the pattern m characters. There are n − m + 1 possible alignments (shifts). At each shift the algorithm compares characters from left to right until the first mismatch or until all m characters match. The basic operation is the comparison of one text character with one pattern character, and every such test, including the one that detects a mismatch, is counted.

```
BruteForceMatch(T[0..n-1], P[0..m-1])
FOR i <- 0 TO n-m DO                  // one shift per iteration
  j <- 0
  WHILE j < m DO
    comparisons++                     // basic operation
    IF T[i+j] != P[j] THEN BREAK
    j <- j + 1
  END WHILE
  IF j = m THEN report match at i
END FOR
```

$$1 \leq \text{comparisons per shift} \leq m \quad \Rightarrow \quad n-m+1 \leq C \leq m(n-m+1)$$

The two extremes are reached when every shift fails at the first character (cost n − m + 1, which is Θ(n)) and when every shift matches m − 1 characters before failing (cost m(n − m + 1), which is Θ(nm)). The experiment designs inputs for each extreme and one realistic case.

► Step 2: The test conditions and the predictions made before execution

Test	Text and pattern	Prediction (made before running)	Algorithmic reason
T1 – little work	Text: 10,000 characters “B”. Pattern: “ABBBBBBBBB” (m = 10)	Exactly n − m + 1 = 9,991 comparisons	The first pattern character “A” never matches the text character “B”, so each shift ends after a single comparison.
T2 – engineered large work	Text: 10,000 characters “A”. Pattern: “AAAAAAAAAB” (m = 10)	Exactly m(n − m + 1) = 10 × 9,991 = 99,910 comparisons	At every shift the first 9 characters match and only the last one fails, so every shift costs the maximum of 10 comparisons and the work is repeated for overlapping windows.
T3 – own choice: realistic log	Text: 10,000 characters of synthetic SSH/CRON log lines. Pattern: “Failed password for root” (m = 24)	About 1.0 to 1.3 comparisons per shift, so roughly 10,000 to 13,000 in total	In ordinary English-like text the first pattern character “F” matches only occasionally, and a match of the second character is rarer still, so most shifts fail at the first or second comparison.
T4 – own choice: random binary text	Text: 10,000 random characters from {0, 1}. Pattern: “1111111110” (m = 10)	About $(2 - 2^{-9}) \times 9{,}991 \approx$ 19,962 comparisons	With only two symbols each comparison matches with probability ½, so the expected comparisons per shift is 1 + ½ + ¼ + … ≈ 2.

For an alphabet of size σ in which the pattern’s leading characters match random text with probability 1/σ each, the expected number of comparisons per shift is a geometric sum, which explains T3 and T4:

$$E[\text{comparisons per shift}] = \sum_{k=0}^{m-1} \sigma^{-k} = \frac{1-\sigma^{-m}}{1-\sigma^{-1}}$$

For σ = 2 this is about 2; for the 26+ symbols that appear in log text it is close to 1.

► Step 3: Observed results

Test	n	m	Shifts n − m + 1	Comparisons observed	Comparisons per shift	Matches found
T1 – best-case structure	10,000	10	9,991	9,991	1.000	0
T2 – engineered worst case	10,000	10	9,991	99,910	10.000	0
T3 – realistic log	10,000	24	9,977	10,747	1.077	10
T4 – random binary	10,000	10	9,991	19,714	1.973	11

Table 6: Character comparisons for the four test conditions.

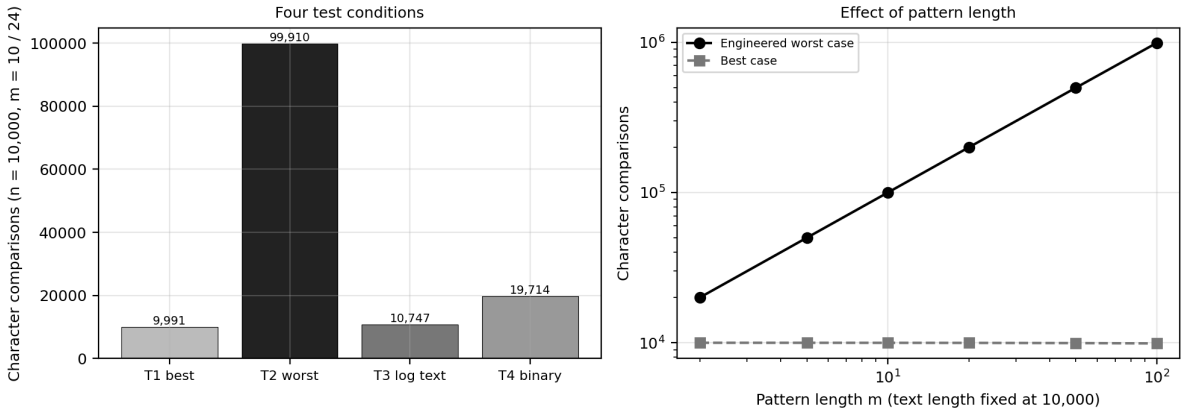


Figure 4: Left – comparisons for the four tests. Right – best and worst case as the pattern length grows (text length fixed at 10,000).

Pattern length m	Best case observed	Predicted n − m + 1	Worst case observed	Predicted m(n − m + 1)
2	9,999	9,999	19,998	19,998
5	9,996	9,996	49,980	49,980
10	9,991	9,991	99,910	99,910
20	9,981	9,981	199,620	199,620
50	9,951	9,951	497,550	497,550
100	9,901	9,901	990,100	990,100

Table 7: Effect of the pattern length on the engineered extremes (n = 10,000).

Text length n	Worst case observed (m = 10)	Predicted 10(n − 9)
1,000	9,910	9,910
2,000	19,910	19,910
4,000	39,910	39,910
8,000	79,910	79,910
16,000	159,910	159,910

Table 8: Effect of the text length on the worst case. Doubling n exactly doubles the work.

► Step 4: Prediction compared with the experimental result

Test	Predicted	Observed	Difference	Verdict
T1	9,991	9,991	0	Exact
T2	99,910	99,910	0	Exact
T3	10,000 – 13,000	10,747	7.7% above one comparison per shift	Within the predicted range
T4	about 19,962	19,714	-1.2%	Close; the small difference is random variation in the generated text

Interpretation: the two engineered tests match the formulas exactly because their behaviour is fully determined by the structure of the input. The two statistical tests match the expected-value reasoning to within about one percent, which shows that the prediction method is sound for realistic data as well as for designed data.

► Inquiry: why the structure matters

Why one input requires substantially more work than another. T1 and T2 have the same text length (10,000) and the same pattern length (10), so they have the same n and m and the same number of shifts, 9,991. Yet T2 needed 99,910 comparisons and T1 needed 9,991, a factor of exactly 10. In T1 each shift is rejected after one comparison. In T2 each shift is rejected only after reading 9 matching characters, and almost all of that reading is repeated, because the window that starts one position later re-reads 8 of the 9 characters that the previous window had just compared. The brute-force algorithm remembers nothing about characters it has already examined.

Which characteristics of the text and pattern create repeated comparisons. The evidence shows four:

- *Long matching prefixes.* The longer the part of the pattern that agrees with the text before the mismatch, the more comparisons are wasted. In T2 the prefix is 9 characters long at every shift.
- *A mismatch near the end of the pattern.* If the pattern's distinguishing character is at the end ("AAAAAAAAAB"), almost all work is done before the failure is found. If it is at the front (T1), the failure is found at once.
- *Repetitive text and a small alphabet.* A single repeated symbol makes every window look alike (T2), and a two-symbol alphabet doubles the cost of random text (T4: about 1.97 comparisons per shift against 1.08 for ordinary log text).
- *Long patterns.* The worst case grows in direct proportion to m (Table 7: 19,998 for m = 2 up to 990,100 for m = 100), while the best case does not depend on m at all (about 10,000).

What this reveals about evaluating an algorithm using only input size. The input size n (together with m) does not determine the running time. For the same n = 10,000 and m = 10 the work ranged over a factor of 10, and for m = 100 the engineered worst case needed 990,100 comparisons against a best case of 9,901, a factor of 100. A statement such as "the algorithm needs n comparisons" would be wrong by up to a factor of m. A correct evaluation must state the best case $\Theta(n)$, the worst case $\Theta(nm)$ and, because ordinary logs behave much like T3, the average case, which is close to n for natural text.

Consequence for log screening. For typical logs the observed cost is only about 8% above the minimum, so the initial brute-force system is entirely acceptable for ordinary data. The risk is adversarial or highly repetitive data: a log that contains long runs of one character, or a signature made of a repeated prefix, can slow the search by a factor up to m. Because a security tool processes data that an attacker may control, an engineer should test the worst case, not only the typical case, before relying on brute force. This is the same principle that Q7 applies to Q4: understand where the work comes from before deciding whether to redesign.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 Marks)

A logistics company operates delivery drones. At a given instant each drone D_i reports its position (x_i, y_i) and the monitoring system must find the two drones that are closest to each other. The brute-force approach is to measure the distance for every pair of drones and keep the smallest. This question designs that solution, derives how many pairs it examines, measures its growth and decides when it stops being practical.

► Step 1: Design questions answered before implementation

Which candidate pairs must be examined? Any two drones could be the closest pair, and nothing about one pair tells us anything about another, so a brute-force solution must examine every pair of distinct drones. A drone is never paired with itself.

How can duplicate pair comparisons be avoided? The distance from D_i to D_j equals the distance from D_j to D_i. Examining both would do every comparison twice. By fixing the order i < j, every unordered pair appears exactly once: the outer loop runs i from 1 to n − 1 and the inner loop runs j from i + 1 to n.

How many unique pairs exist for n drones? Drone 1 is paired with the n − 1 drones after it, drone 2 with n − 2, and so on, down to drone n − 1 which is paired with 1. Therefore:

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1 = \sum_{k=1}^{n-1} k = \frac{(n-1)n}{2} = \binom{n}{2}$$

The same result follows from counting ordered pairs: each of the n drones has n − 1 partners, giving n(n − 1) ordered pairs, and every unordered pair has been counted twice, so there are n(n − 1)/2 of them. The number of pairs is $\Theta(n^2)$: doubling the number of drones multiplies it by almost exactly four.

n	2	3	4	5	10	100	1,000	10,000	1,000,000
n(n−1) /2	1	3	6	10	45	4,950	499,500	49,995,000	4.99999×10^{11}

The distance is $d = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$. Because the square root is increasing, comparing squared distances gives the same winner, so the implementation avoids the square root and takes it once, for the final answer. One distance calculation is performed for each pair, so “pairs examined” and “distance calculations” are equal for brute force.

```
BruteForceClosestPair(D[1..n])
best <- infinity;  pairs <- 0;  dists <- 0
FOR i <- 1 TO n-1 DO
  FOR j <- i+1 TO n DO
    pairs++
    d2 <- (xi - xj)^2 + (yi - yj)^2        // squared distance
    dists++
    IF d2 < best THEN best <- d2;  closest <- (i, j)
  END FOR
END FOR
RETURN closest, sqrt(best)
```

► Step 2: Experiment

Drone positions were generated uniformly at random in a 10,000 × 10,000 square (fixed seed). Eight input sizes were used, doubling each time from 100 to 12,800. The counters record pairs examined and distance calculations; time is the best of three runs for the small sizes and a single run for the largest.

n	n(n−1)/2 (formula)	Pairs examined	Distance calculations	Time (s)	Pairs ratio to previous row	Time ratio to previous row
100	4,950	4,950	4,950	0.0005	–	–
200	19,900	19,900	19,900	0.0019	4.02	3.80
400	79,800	79,800	79,800	0.0078	4.01	4.11
800	319,600	319,600	319,600	0.0328	4.01	4.21
1,600	1,279,200	1,279,200	1,279,200	0.1303	4.00	3.97
3,200	5,118,400	5,118,400	5,118,400	0.5897	4.00	4.53
6,400	20,476,800	20,476,800	20,476,800	2.5925	4.00	4.40
12,800	81,913,600	81,913,600	81,913,600	10.1717	4.00	3.92

Table 9: Brute-force closest pair, measured growth.

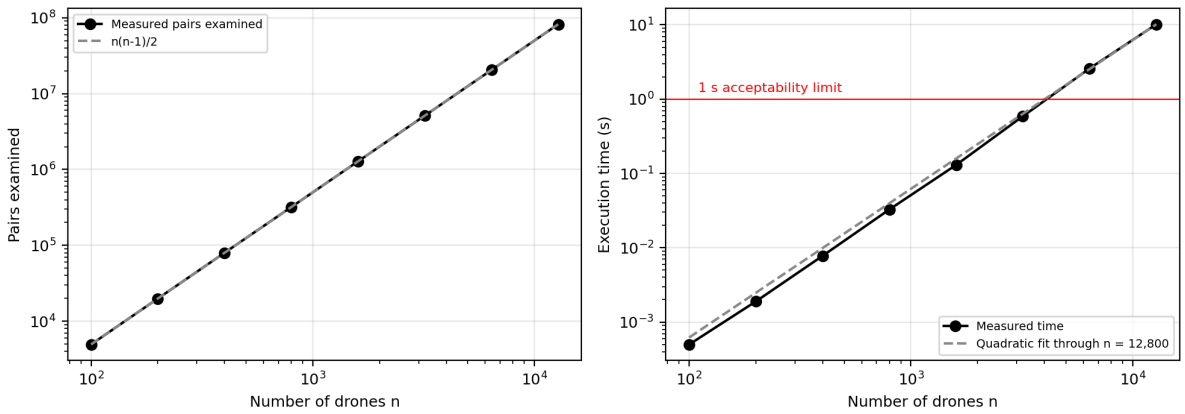


Figure 5: Left – pairs examined against the formula n(n−1)/2. Right – execution time against a quadratic curve, with the 1-second acceptability limit.

► Step 3: Theoretical growth compared with the experiment

- **The counts match the derivation exactly.** For every size the number of pairs examined equals n(n − 1)/2 and equals the number of distance calculations. For example, n = 12,800 gives 81,913,600 pairs.
- **Growth is quadratic.** Each doubling of n multiplies the pair count by 4.02 at n = 200 and by values closer and closer to 4 afterwards (4.00 at n = 12,800), as the formula n(n−1)/2 predicts, since 4(n−1)/(n−2) tends to 4.
- **Time follows the same curve.** The time ratio per doubling is close to 4 for the larger sizes (4.5, 4.4 and 3.9 for the last three doublings). The ratios for smaller n are noisier, because very short runs of a few milliseconds are affected by timer resolution, and for large n the time per pair rises slightly (about 0.10 µs per pair at n = 800 against 0.12 µs at n = 12,800), probably

because the working data no longer fit in the processor cache. The operation counts, not the times, are the primary evidence, and the counts are exact.

► Scalability inquiry: acceptable response time

Acceptability criterion. The monitoring system receives a new position report from each drone every second, so I define an acceptable response time as **at most 1 second** for one complete closest-pair computation in this experimental environment: a result that arrives after the next report has been received is already out of date and cannot be used to avoid a collision. Tighter and looser limits are also shown, because the criterion is a design choice.

$$T(n) \approx k \cdot \frac{n(n-1)}{2}, \qquad k = \frac{10.17 \text{ s}}{81{,}913{,}600} \approx 1.24 \times 10^{-7} \text{ s per pair}$$

Response-time limit	Largest n (from the model)	Measured evidence
0.1 s	about 1,270 drones	n = 800 took 0.033 s and n = 1,600 took 0.130 s
1 s (chosen criterion)	about 4,014 drones	n = 3,200 took 0.59 s (within limit); n = 6,400 took 2.59 s (exceeds limit)
10 s	about 12,692 drones	n = 12,800 took 10.17 s

Conclusion. With a 1-second limit the brute-force approach is comfortable up to about 3,000 drones, reaches its limit at roughly **4,014 drones**, and is clearly impractical beyond about 6,400 drones (more than 2.5 s). Because the cost is quadratic, a 100-fold faster computer would only extend the limit by a factor of 10, to about 40,000 drones, so hardware cannot rescue the approach for fleets of hundreds of thousands of drones.

► Origin of the inefficiency and what a redesign must achieve

The measurements show where the cost comes from. Every distance calculation is independent of the others, so nothing computed for one pair is used for another. The algorithm also makes no use of the geometry: it measures the distance between drones that are 9,000 metres apart with exactly the same effort as between drones that are 10 metres apart, although the former can never be the closest pair. A second count, reported in Q7, makes this concrete: for n = 3,200 drones only 2,563 of the 5,118,400 pairs (0.05%) are close enough along one axis to be capable of beating the final minimum distance, so about 99.95% of the distance calculations are wasted.

A redesigned strategy would therefore need to:

- avoid examining pairs that cannot possibly be the closest, by exploiting spatial locality (for example by ordering or partitioning the drones so that near neighbours are found together);
- reuse information already computed, in particular the best distance found so far, as a bound that eliminates candidates cheaply;
- keep the answer identical to the brute-force answer, since a collision monitor cannot tolerate a missed pair.

Interpretation: brute force is a sound baseline for fleets of a few thousand drones and has the advantage of being obviously correct, but its quadratic growth makes it the first component to fail as the fleet grows. Q7 develops a redesign that follows exactly the three requirements above.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 Marks)

Monitoring stations are placed throughout a wildlife reserve. Each station is a point $P_i = (x_i, y_i)$ and the officers need the outer boundary formed by the stations, which is the convex hull of the points. Only one geometric principle is supplied: *a line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.* The algorithm is derived from this principle below, step by step.

► Step 1: From the principle to a computational procedure

How do two candidate points define a line? Take two stations $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$. Every point (x, y) on the line through them satisfies $(y_2 - y_1)(x - x_1) = (x_2 - x_1)(y - y_1)$. Collecting terms gives the line in the form ax + by = c:

$$a = y_2 - y_1, \qquad b = x_1 - x_2, \qquad c = x_1 y_2 - y_1 x_2$$

Check: substituting P_1 gives $a \cdot x_1 + b \cdot y_1 = x_1 y_2 - x_1 y_1 + x_1 y_1 - x_2 y_1 = c$, and similarly for P_2.

How can the position of another point relative to the line be determined? Define the value s for a third point $P_k = (x_k, y_k)$:

$$s_k = a\,x_k + b\,y_k - c$$

The expression is zero exactly on the line. Moving perpendicular to the line changes s continuously and in one direction only, so s > 0 on one side of the line and s < 0 on the other. Only the sign matters, so no division or square root is needed and, for integer coordinates, the test is exact. (With this choice of a, b, c, s > 0 means that P_k lies to the right of the direction $P_1 \rightarrow P_2$.)

How must the remaining points be examined? Every point other than P_i and P_j must be tested, because a single point on the wrong side is enough to show that the line is not on the boundary. The scan can therefore stop at the first point whose sign is opposite to an earlier sign (early exit).

What condition identifies a potential hull edge? The segment P_iP_j is accepted when all remaining points lie in one closed half-plane of the line: either $s_k \geq 0$ for all k or $s_k \leq 0$ for all k. Points with $s_k = 0$ are collinear with the line. They need extra care: a collinear point lying between P_i and P_j is harmless (it lies on the edge), but a collinear point outside the segment means that P_i or P_j is not an extreme point of that side, so the longer segment is the true edge and this one is rejected.

How will the final boundary be constructed? Every accepted edge is directed so that all other points lie on its left. Because the hull is a convex polygon, each hull vertex then has exactly one outgoing edge, so the edges can be stored as a successor table and followed from any start vertex until the start is reached again. This gives the vertices in counter-clockwise order without any sorting.

► Step 2: Pseudocode designed from the decision rule

```
BruteForceHull(Points)
 1  pts <- distinct points of Points;  n <- |pts|
 2  IF n < 3 THEN RETURN pts
 3  edges <- empty list
 4  FOR i <- 0 TO n-2 DO
 5    FOR j <- i+1 TO n-1 DO                  // candidate line through pts[i], pts[j]
 6      a <- yj - yi;  b <- xi - xj;  c <- xi*yj - yi*xj
 7      positive <- false;  negative <- false;  ok <- true
 8      FOR k <- 0 TO n-1, k != i, k != j DO    // every remaining point
 9        s <- a*xk + b*yk - c                  // side test
10        IF s > 0 THEN positive <- true
11        ELSE IF s < 0 THEN negative <- true
12        ELSE IF pts[k] is outside segment (pts[i], pts[j]) THEN ok <- false
13        IF (positive AND negative) THEN ok <- false
14        IF NOT ok THEN BREAK                  // early exit: not a boundary edge
15      END FOR
16      IF ok THEN add (i, j, positive, negative) to edges
17    END FOR
18  END FOR
19  IF no point lies off any accepted line (all collinear) THEN RETURN the two end points
20  FOR each accepted edge (i, j):
21    IF positive AND NOT negative THEN swap(i, j)   // make all other points lie to the left
22    next[i] <- j
23  START at any vertex v; REPEAT output pts[v]; v <- next[v] UNTIL v returns to START
24  RETURN the output list (counter-clockwise hull vertices)
```

Special cases handled explicitly: duplicate points are removed first (a duplicate pair would give a = b = c = 0, so every point would produce s = 0 and be treated as collinear); fewer than three distinct points are returned unchanged; and if all points are collinear every pair is accepted, so the hull degenerates to the two extreme points (line 19).

► Step 3: Test configurations

- **Test 1 – boundary and interior points.** 30 random integer points in the square [0, 100] × [0, 100]. Most points are interior, so the algorithm must reject nearly all of the 435 candidate lines.
- **Test 2 – designed to challenge the algorithm.** A 5 × 5 grid of integer points plus two duplicated points, (2, 2) and (0, 0). The grid puts 16 points on the boundary, 12 of them on the edges but not at the corners, so every edge of the hull has three collinear points on it. It also contains horizontal and vertical lines (a = 0 or b = 0), which are special for many hand-written implementations, and the duplicates test the pre-processing.
- **Test 3 – every outer point is a vertex.** 12 points on a circle (rounded to integers) with 5 interior points. All 12 outer points must be reported and none of the interior points may be.

Test	Input points	Distinct points	Hull vertices found	Candidate pairs examined	Side tests	Matches independent check
1 – random	30	30	9	435	4,014	Yes
2 – grid with duplicates	27	25	4	300	2,368	Yes
3 – circle + interior	17	17	12	136	881	Yes

Table 10: Results of the three test configurations. The independent check is a separate monotone-chain implementation used only for verification.

Hull of Test 1 (counter-clockwise, starting at the leftmost point): (0, 37), (8, 2), (27, 3), (99, 22), (93, 47), (73, 90), (39, 97), (37, 97), (13, 92). **Hull of Test 2:** (0, 0), (4, 0), (4, 4), (0, 4) – only the four corners; the twelve collinear boundary points are correctly not reported as vertices, and the duplicates did not create extra edges. **Hull of Test 3:** the 12 circle points, none of the 5 interior points.

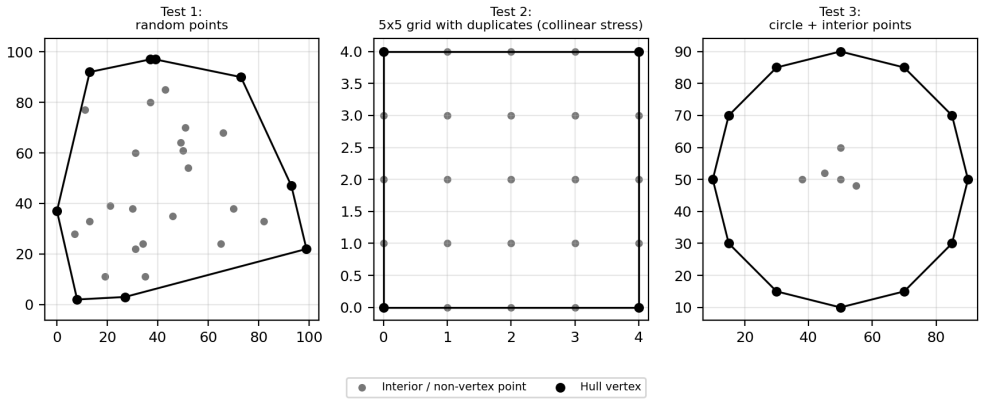


Figure 6: Boundaries produced by the brute-force algorithm. Black dots are hull vertices; grey dots are interior or collinear non-vertex points.

► Step 4: Computational analysis

The algorithm examines each of the n(n − 1)/2 pairs and, for each, tests up to n − 2 other points. In the worst case no pair is rejected early, giving:

$$W(n) = \frac{n(n-1)}{2}(n-2) \in \Theta(n^3)$$

In practice the early exit stops most scans quickly. The measured side-test counts for two layouts, one with few hull vertices (random points) and one in which every point is a hull vertex (points on a circle, stored in angular order) are shown below. Both results were checked against the independent monotone-chain hull for every size.

n	Pairs n(n−1)/2	Worst case n(n−1)(n−2)/2	Random: side tests	% of worst	Hull size	Circle: side tests	% of worst	Hull size	Random time (s)
25	300	6,900	2,537	36.8	7	3,127	45.3	25	0.001
50	1,225	58,800	17,588	29.9	10	23,127	39.3	50	0.004
100	4,950	485,100	118,680	24.5	11	176,252	36.3	100	0.025
200	19,900	3,940,200	835,408	21.2	13	1,372,502	34.8	200	0.211
400	79,800	31,760,400	6,195,376	19.5	19	10,825,002	34.1	400	1.537

Table 11: Growth of computational work with the number of points.

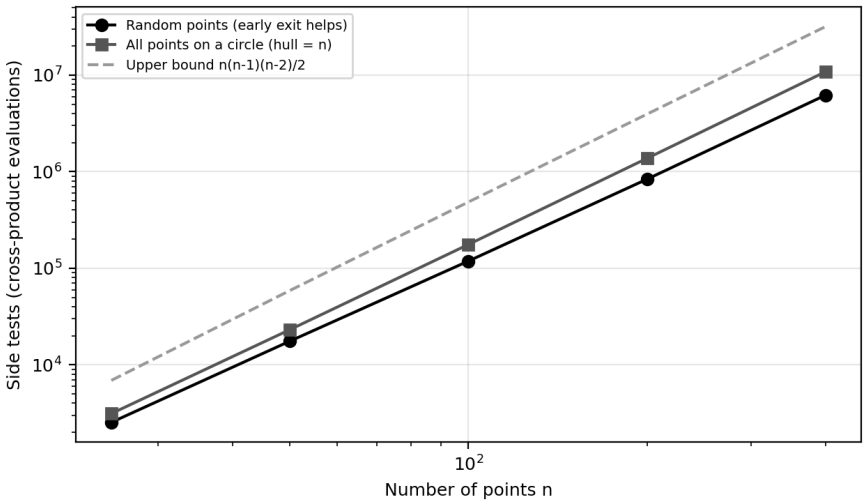


Figure 7: Side tests against n for random and circular layouts, with the worst-case bound (both axes logarithmic).

- **Number of pairs:** exactly $n(n-1)/2$ in every row, so this part grows as $\Theta(n^2)$.
- **Cubic growth of the tests:** doubling n from 200 to 400 multiplied the side tests by 7.42 for random points and 7.89 for the circle, close to the factor 8 expected for n^3. The measured constants are about 0.10 n^3 (random) and 0.17 n^3 (circle), against 0.5 n^3 for the worst case.
- **Effect of the layout:** the circle needs more tests than random points (10.8 million against 6.2 million at n = 400) because every point is a hull vertex. Each of the n accepted edges needs a full scan of the other $n - 2$ points, and rejected chords between nearby vertices are discovered late. Random points have few hull vertices (19 of 400), so most pairs are rejected after only a few tests.
- **Time:** n = 400 took 1.54 s for random points and 2.64 s for the circle. Extrapolating the cubic trend, 4,000 points would need about 1,000 times longer, about 25 to 45 minutes, so this algorithm suits tens or a few hundred monitoring stations, not tens of thousands.

► Correctness inquiry

Why does the geometric test provide evidence that an accepted edge belongs to the outer boundary? The argument uses only geometry, not the program output.

- **Soundness (an accepted edge is on the boundary).** The convex hull H of the points is the smallest convex set that contains all of them. Suppose line L through P_i and P_j has all other points in one closed half-plane. That half-plane is convex and contains every point, so it contains the hull: H lies entirely on one side of L (or on it). But P_i and P_j are points of H, and they lie on L. A point of a convex set that lies on a supporting line is a boundary point of the set, and the whole segment P_iP_j lies in $H \cap L$, which is a boundary face of H. Hence the segment cannot pass through the interior of the hull; it belongs to the boundary. The test for collinear points ensures that the accepted segment is the whole face and not just part of it.
- **Why a rejected line is not a boundary edge.** If points exist strictly on both sides of L, then L cuts through the hull, because the hull contains points on both sides of it and is convex, so it contains the segment between them, which crosses L. Part of the segment P_iP_j is then inside the hull rather than on its boundary, so the pair cannot be a hull edge. The early exit is therefore not an approximation but a proof of rejection.

- **Completeness (every hull edge is accepted).** A hull edge joins two hull vertices, which are input points. The line containing the edge is a supporting line of the convex hull, so all points lie on one side of it, and the test accepts it. Collinear points on the edge lie between its end points, so they do not cause rejection; collinear points outside it cannot exist because the edge ends at hull vertices.
- **Why the boundary is closed.** Each hull vertex is the end of exactly one hull edge and the start of exactly one (after orienting every edge with all points on the left), so following the successor table visits each vertex once and returns to the start.

This reasoning also explains the three tests: in Test 2 each of the four candidate edges has all $25 - 2$ other points on one side or on the line, with the collinear boundary points between the corners, so the edge is accepted as a single segment from corner to corner; every interior chord has points on both sides and is rejected. **Interpretation:** the decision rule is a direct translation of a geometric fact, and its correctness follows from convexity. The price of this simplicity is $\Theta(n^3)$ work, which makes the method a good reference implementation and a poor production algorithm for large point sets.

Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

A helicopter carrying emergency supplies has a strict payload limit W. Each package has an ID, a weight w_i and a humanitarian value v_i. The planner must choose the combination of packages whose total weight does not exceed W and whose total value is as large as possible. An exhaustive-search strategy examines every possible combination, so it always finds the true optimum, but its cost grows very quickly with the number of packages. This question designs the strategy, verifies it on a small instance, predicts and measures its growth, and decides how large a problem it may be trusted with.

► Step 1: Design of the exhaustive search

How will candidate package combinations be represented? Each package is either loaded or left behind, so a combination is a choice of "in" or "out" for each of the n packages. A combination is represented by an n-bit binary number (a bit mask): bit k is 1 if package k is loaded. The mask 0101 for four packages means that packages 1 and 3 are loaded (bit 0 is package 1). Every combination corresponds to exactly one mask and every mask to exactly one combination, so the masks 0 to $2^n - 1$ are all the candidates, including the empty load.

How will candidate solutions be generated? A counter runs from 0 to $2^n - 1$ and each value is read as a mask. The clearest version recomputes the weight and value of each mask from its set bits. A faster version, used for the large experiments, visits the same 2^n masks in Gray-code order, in which each mask differs from the previous one by exactly one bit; the total weight and value are then updated by adding or removing one package, which costs O(1) per candidate instead of O(n). Both versions examine all candidates and prune nothing.

How will feasibility be checked? A candidate is feasible if its total weight is at most W.

How will feasible solutions be evaluated? The value of a feasible candidate is the sum of the values of its packages.

How will the best solution be retained? Two variables, bestValue and bestMask, are kept. When a feasible candidate has a value greater than bestValue, both are replaced. If two candidates have the same value the first one found is kept.

```
ExhaustiveLoad(w[0..n-1], v[0..n-1], W)
bestValue <- -1;  bestMask <- 0;  candidates <- 0;  feasible <- 0
FOR mask <- 0 TO 2^n - 1 DO                      // generate every combination
  candidates++
  totalW <- 0;  totalV <- 0
  FOR k <- 0 TO n-1 DO
    IF bit k of mask = 1 THEN totalW <- totalW + w[k];  totalV <- totalV + v[k]
  END FOR
  IF totalW <= W THEN                            // feasibility check
    feasible++
    IF totalV > bestValue THEN                   // evaluate and retain the best
      bestValue <- totalV;  bestMask <- mask
  END IF
END FOR
RETURN bestMask, bestValue, candidates, feasible
```

► Step 2: Correctness check on a small instance

A four-package instance was chosen so that all $2^4 = 16$ combinations can be listed and checked by hand. The payload limit is W = 10. The packages are P1 (weight 7, value 42), P2 (weight 3, value 12), P3 (weight 4, value 40) and P4 (weight 5, value 25).

Mask (P4 P3 P2 P1)	Packages loaded	Total weight	Total value	Feasible (≤ 10)?
0000	none	0	0	Yes
0001	P1	7	42	Yes
0010	P2	3	12	Yes
0011	P1, P2	10	54	Yes
0100	P3	4	40	Yes
0101	P1, P3	11	82	No
0110	P2, P3	7	52	Yes
0111	P1, P2, P3	14	94	No
1000	P4	5	25	Yes
1001	P1, P4	12	67	No
1010	P2, P4	8	37	Yes
1011	P1, P2, P4	15	79	No
1100	P3, P4	9	65	Yes
1101	P1, P3, P4	16	107	No
1110	P2, P3, P4	12	77	No
1111	P1, P2, P3, P4	19	119	No

Table 12: All 16 candidates for the verification instance (the optimum is shaded).

Independent verification. Reading the table by hand: 9 of the 16 combinations are feasible; the heaviest feasible loads are P1 + P2 (weight 10, value 54) and P3 + P4 (weight 9, value 65); every load that contains both P1 and P3 weighs at least 11 and is infeasible; and no feasible load exceeds a value of 65. The optimum is therefore **P3 + P4 with weight 9 and value 65**. The program returned bestValue = 65 with packages P3, P4, examined 16 candidates and found 9 feasible ones, all matching the hand analysis. The Gray-code version returned the same value (65), and an independent dynamic-programming computation also gave 65.

The check was repeated on 300 randomly generated instances with 1 to 14 packages and random limits; the plain search, the Gray-code search and the dynamic-programming reference gave the same optimal value in **300 of 300** cases. (The dynamic-programming method is used only as a checker; it is not part of the exhaustive-search solution.)

► Step 3: Growth prediction before experimentation

Each package contributes an independent choice of two options, so by the product rule the number of candidates is:

$$N(n) = 2 \times 2 \times \cdots \times 2 \ (n \text{ factors}) = 2^n$$

Adding one package doubles the solution space: every existing combination can be extended by either leaving the new package out or putting it in, so the new space contains the old space twice. Consequently:

- the number of candidates doubles with each extra package and is multiplied by 2^{Δ} for Δ extra packages (by 4 for 2 more packages, by 1,024 for 10 more);
- with a constant cost per candidate (the Gray-code version), the running time should also double with each extra package;
- the plain version recomputes sums over n packages for each candidate, so I predict a cost proportional to $n \cdot 2^n$, slower than the Gray-code version by a factor that grows with n;
- about half of the candidates should be feasible when W is half of the total weight, because the total weight of a random subset is symmetric about half the total weight.

► Step 4: Experimental validation of the prediction

For each problem size n a random instance was generated (weights 5–40, values 10–100, payload limit equal to half of the total weight, fixed seed). Eight sizes from 10 to 24 packages were run, followed by n = 25 and n = 26 to locate the deployment boundary. The optimum was also checked against dynamic programming at every size; it matched in all ten cases.

n	Candidates 2^n (predicted)	Candidates examined	Feasible candidates	Best value	Time (s)	Time ratio to previous size	ns per candidate
10	1,024	1,024	514	322	0.0002	–	195
12	4,096	4,096	2,059	407	0.0007	3.50	171
14	16,384	16,384	8,246	511	0.0029	4.14	177
16	65,536	65,536	32,988	637	0.0117	4.03	179
18	262,144	262,144	131,072	683	0.0561	4.79	214
20	1,048,576	1,048,576	524,288	820	0.1863	3.32	178
22	4,194,304	4,194,304	2,110,889	835	0.9125	4.90	218
24	16,777,216	16,777,216	8,440,288	887	2.9950	3.28	179
25	33,554,432	33,554,432	16,777,216	990	6.6283	2.21	198
26	67,108,864	67,108,864	33,554,432	993	13.0252	1.97	194

Table 13: Exhaustive search, measured growth (the ratio column compares each size with the previous size in the table, which is two packages smaller up to n = 24 and one package smaller for n = 25 and 26).

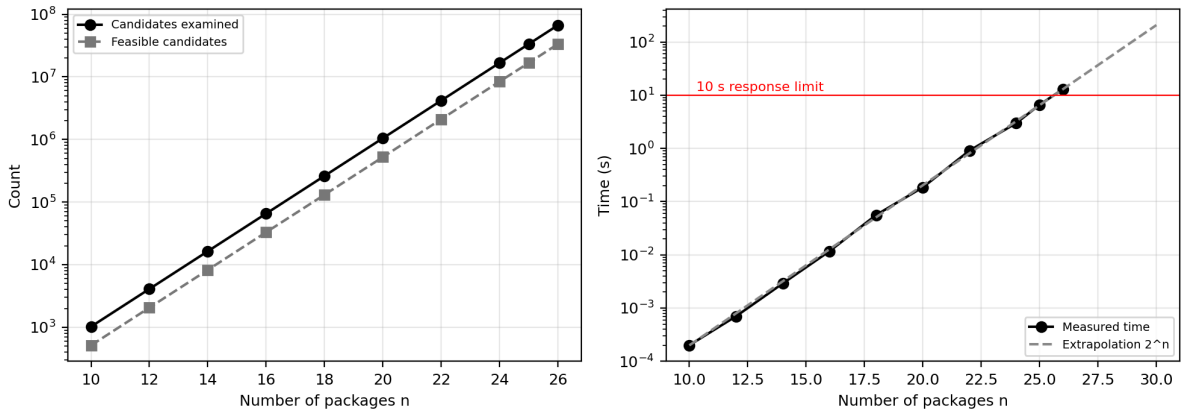


Figure 8: Left – candidates examined and feasible candidates (logarithmic axis). Right – execution time with a 2^n extrapolation and the 10-second limit.

Comparison of prediction and result.

- **Counts:** the candidates examined equal 2^n exactly for every n (for instance 16,777,216 for n = 24 and 67,108,864 for n = 26). The prediction is confirmed with no error.
- **Feasible fraction:** the feasible candidates are close to half of all candidates at every size (524,288 of 1,048,576 for n = 20, 16,777,216 of 33,554,432 for n = 25), as predicted by the symmetry argument.
- **Time doubles per package:** from n = 25 to n = 26 the time rose from 6.63 s to 13.03 s, a ratio of 1.97; from n = 24 to n = 25 the ratio was 2.21. For each pair of sizes that differ by two packages the predicted ratio is 4; the observed ratios (between 3.3 and 4.9) vary because of timer resolution at short times and memory effects, but the cost per candidate stays between about 175 and 220 ns for $n \geq 14$, so the time follows 2^n within about 20%.
- **Plain versus Gray-code search:** both versions returned the same optimum at n = 10 to 16. The plain version took 0.0744 s at n = 16 against 0.0117 s for the Gray-code version, 6.4 times longer, and the gap widens with n (3.0 times at n = 10), which agrees with the $n \cdot 2^n$ against 2^n prediction.

► Inquiry: why do a few extra packages cause such a large increase in effort?

The reason is that the solution space grows **exponentially**, not by a fixed amount, with the number of packages. A new package does not add one more candidate or n more candidates; it creates a second copy of the entire existing space, namely every old combination with the new package added. The space therefore doubles each time, and a small number of additional packages multiplies the work by a very large factor. Compare this with polynomial growth. Going from 20 to 30 packages multiplies a quadratic cost by $(30/20)^2 = 2.25$, but multiplies the exhaustive search by $2^{10} = 1{,}024$. Using the cost per candidate measured at n = 26 (about 194 ns), the table shows what each additional group of packages means in time.

Packages n	Candidates 2^n	Growth compared with n = 20	Time (measured, or estimated at 194 ns per candidate)	Source of the time
20	1,048,576	× 1	186 ms	measured
24	16,777,216	× 16	3.0 s	measured
26	67,108,864	× 64	13.0 s	measured
30	1,073,741,824	× 1,024	3.5 minutes	extrapolated
35	3.44×10^{10}	× 32,768	111.1 minutes	extrapolated
40	1.10×10^{12}	× 1,048,576	2.5 days	extrapolated
50	1.13×10^{15}	× 1,073,741,824	6.9 years	extrapolated
60	1.15×10^{18}	× 1,099,511,627,776	7,103.8 years	extrapolated

Interpretation: doubling the number of packages from 20 to 40 does not double the effort or square it; it multiplies it by about a million (2^{20}), turning a fraction of a second into more than two days. This is characteristic of combinatorial problems: the number of combinations, not the length of the input, determines the work, and no increase in hardware speed can offset a doubling per package for long.

► Deployment decision

Response-time requirement. I assume that the airlift planner must produce a loading decision within **10 seconds**. The justification is operational: loading decisions are repeated whenever the package list changes (new packages arrive, others are cancelled), so a planner who waits more than about ten seconds after each change loses the ability to explore alternatives, while shorter limits would unnecessarily restrict the problem size. This is a modelling assumption for the experiment; the real limit would be set by the operations team.

Response-time limit	Largest n tested within the limit (measured)	First size that fails (measured or extrapolated)
1 s	22 packages (0.91 s)	23 packages (extrapolated: about 1.8 s)
10 s (chosen)	25 packages (6.6 s)	26 packages (13.0 s)
60 s	28 packages (extrapolated: about 52 s)	29 packages (extrapolated: about 104 s)

Decision. For the 10-second limit the measured boundary is **25 packages** (6.6 s), because n = 26 needs 13.0 s. I would deploy the solution for up to **24 packages** (3.0 s measured), keeping a safety factor of more than three, because the field computer may be slower than the test machine, may be running other software, and the time of a particular instance can vary. 25 packages is the absolute limit and 26 or more must not be accepted. For a larger number of packages the planner needs a different method, such as pruning a search tree when the weight limit is exceeded or exploiting the structure of the capacity, rather than a faster computer. Because a request that exceeds the limit would take hours or years rather than seconds, the deployed system should check n before starting and refuse, or switch method, when n exceeds the boundary.

Q7 – Diagnose, Redesign and Validate (13 Marks)

This design challenge reviews the evidence of Q1–Q6, chooses one solution whose computational limitation becomes serious as the problem grows, and redesigns it by understanding the cause of the limitation rather than by naming a known algorithm.

► Choice of solution to redesign

Each solution was reviewed against its own scalability evidence:

Question	Observed limitation	Evidence	Suitable for redesign?
Q1 Sorting	Quadratic for random data, but linear for nearly sorted data with Insertion Sort	25,009 comparisons at n = 1,000 for 95% sorted data	No – already resolved by choosing Insertion Sort
Q2 Sequential search	Linear per search; acceptable when n · q is small	0.26% CPU load for 20 facilities	No – the remedy is a standard index and was already given
Q3 String matching	Θ(nm) only for engineered input; about n for real logs	1.08 comparisons per shift on log text	Possible, but the typical case is already efficient
Q4 Closest pair	Quadratic growth; 1-second limit reached at about 4,000 drones	20,476,800 pairs at n = 6,400, 2.6 s	**Yes** – grows with fleet size, and only a tiny fraction of the work is useful
Q5 Convex hull	Cubic growth; 1.5 s at only 400 points	6,195,376 side tests at n = 400	Possible, but the redesign would amount to a different algorithm
Q6 Exhaustive search	Exponential growth; limit at 25 packages	2^n candidates	Possible, but any real improvement changes the algorithm class

Selected solution: Q4 – brute-force closest pair of delivery drones. The reasons are that fleet sizes grow naturally, so its limit (about 4,000 drones for a 1-second response) is a real operating problem; that its cost has a single clear source that the measurements already point to; and that the redesign can be validated very precisely, because the correct answer is known from the brute-force solution and the work is counted exactly.

► A. Diagnose the computational bottleneck (2 marks)

Where is most computational effort spent? Essentially all of it (every counted operation) is in the inner loop, where one squared distance is computed for each of the n(n−1)/2 pairs. The only other work is the comparison with the current best, which happens once per pair as well. There is no set-up cost and no output cost that matters, so the distance calculation and comparison in the inner loop *is* the bottleneck.

Which operation grows significantly with input? The number of distance calculations. From Table 9, doubling n from 6,400 to 12,800 quadrupled it from 20,476,800 to 81,913,600, and the time rose from 2.59 s to 10.17 s.

Is work being repeated unnecessarily? No pair is measured twice, so the algorithm does not repeat identical work. The waste is of a different kind: most of the work is spent on pairs that cannot possibly be the answer. To measure this, I counted for each n the pairs whose horizontal gap $|x_i - x_j|$ is smaller

than the final minimum distance d_{min}. Because the Euclidean distance is never smaller than the horizontal gap, a pair with a gap of d_{min} or more cannot be closer than d_{min}.

n	Total pairs examined by brute force	Pairs with $\|\Delta x\| < d_{min}$ (could possibly win)	Share of pairs that could win	Share of the work that is wasted
400	79,800	453	0.568%	99.432%
1,600	1,279,200	861	0.067%	99.933%
3,200	5,118,400	2,563	0.050%	99.950%

Table 14: Diagnosis – how many of the pairs examined by brute force could have been the closest pair (random drones).

Which characteristic of the original algorithm causes the limitation? The algorithm treats every pair as equally plausible and examines them in an arbitrary order, so it ignores two pieces of information that are available at no cost: the *ordering* of the drones in space (nearby drones are close in coordinates) and the *best distance found so far*, which sets an upper limit for any pair that could still improve the answer. Because the computation does not use these, its cost depends only on n and not on the geometry of the problem. The conclusion is that the cost grows with $n(n-1)/2$ although at n = 3,200 only 0.05% of the pairs are able to matter.

► B. Define the design objective (2 marks)

The redesign must make the cost depend on how many pairs are *candidates* rather than on how many pairs *exist*. The measurable objectives are:

- **Correctness constraint:** the redesigned solution must return exactly the same minimum distance as the brute-force solution on every test (tested on all data sets and on 400 additional random cases including ties and duplicates).
- **Primary objective – avoid examining every candidate:** on uniformly random drones, at most 20n distance calculations instead of $n(n-1)/2$. For n = 3,200 this means at most 64,000 against 5,118,400, a reduction of at least 98.7%.
- **Growth objective:** the number of distance calculations should grow close to linearly with n, so that doubling n multiplies it by less than 2.5 rather than by 4.
- **Operating objective:** extend the size that can be handled in 1 second from about 4,000 drones to at least 100,000 drones.
- **Robustness objective:** identify the kinds of input for which the objectives are not met, so that the limitation of the redesign is known.

► C. Design the revised computational strategy (4 marks)

What has been changed. The all-pairs double loop is replaced by a *sorted sweep with a shrinking search window*. The drones are first sorted along one coordinate axis. Each drone is then compared only with the following drones whose coordinate along that axis lies within the current best distance.

What property is exploited. For any two points, the Euclidean distance is at least as large as the difference of their coordinates along either axis:

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \geq |x_i - x_j|$$

Let δ be the smallest distance found so far. If a drone j has $|x_j - x_i| \geq \delta$, then the pair (i, j) cannot improve on δ. If the drones are sorted by x, all later drones have even larger x-gaps, so the scan for drone i can stop completely at the first drone whose gap is at least δ. This replaces "examine all n − i later drones" with "examine only the drones inside a window of width δ". A small δ means a small window, and δ becomes small quickly because the nearest neighbours are examined first.

```
SweepClosestPair(D[1..n])
 1  axis <- the coordinate (x or y) with the larger spread (max - min)
 2  Q <- D sorted by that coordinate                       // preprocessing, O(n log n)
 3  best <- infinity
 4  FOR i <- 1 TO n-1 DO                                    // seed the bound with adjacent pairs
 5     d2 <- squared distance(Q[i], Q[i+1]);  dists++
 6     best <- min(best, d2)
 7  END FOR
 8  FOR i <- 1 TO n-1 DO
 9     FOR j <- i+1 TO n DO
10        gap <- Q[j].axis - Q[i].axis
11        IF gap^2 >= best THEN BREAK                       // no later j can win either
12        d2 <- squared distance(Q[i], Q[j]);  dists++
13        IF d2 < best THEN best <- d2;  closest <- (i, j)
14     END FOR
15  END FOR
16  RETURN closest, sqrt(best)
```

Why it is correct. A pair is skipped only when its axis gap squared is at least the current best squared distance. The pair's true squared distance is at least that gap squared, so it cannot be smaller than best. Because the array is sorted, the break on line 11 also skips only pairs with equal or larger gaps. Hence no pair that could improve the answer is ever skipped, and the result equals the brute-force result. Equal values (ties) cannot hide a better pair because the comparison uses ≥ only to skip pairs that could at best tie.

Operations avoided and reduced. All pairs whose axis gap is not smaller than the current best are avoided, which at n = 3,200 is more than 99.9% of all pairs. The cost of the scan for each i is one comparison of the gap (the "gap check") for each pair actually examined plus one for the pair that ends the scan.

Preprocessing. Yes: sorting costs Θ(n log n) comparisons. The seeding loop (lines 4–7) adds n − 1 distance calculations, whose purpose is to obtain a small δ before the main scan starts; without it the first drone would scan all the others.

Additional memory. Θ(n) for the sorted copy of the drone list (it could be sorted in place if the original order is not needed).

New trade-offs created.

- The cost is no longer independent of the data. If many drones fall inside a window of width δ along the sweep axis, the scan degrades toward quadratic. A set of drones that all have the same x-coordinate defeats a sweep along x, which is why the design chooses the axis with the larger spread.
- A configuration in which the points are spread along both axes but concentrated in thin lines (for example a plus-shaped formation) can still put about n/2 points into one window; the axis choice cannot fix this. This is a genuine limitation of the design and is tested below.
- The design needs to be correct about ties and equal coordinates, which is why the pruning test uses ≥ and the result is checked against brute force.

- More code and more reasoning are needed than for brute force, and the answer depends on a sort.

► D. Predict before implementing (1 mark)

The following predictions were written before the redesigned program was run.

- **Distance calculations (performance measure expected to improve):** the seeding loop contributes n − 1. After seeding, δ is of the order of the typical spacing between nearest neighbours, which for n uniformly random points in a square of side L is of the order L/n, so a window of that width contains only about one other drone on average. I therefore predict roughly **2n** distance calculations in total (between n and 6n), for example about 6,400 for n = 3,200 against 5,118,400, a reduction of about 800 times.
- **Growth:** the counts should grow almost linearly, so doubling n should multiply them by about 2, not 4; execution time should grow like n log n because of the sort.
- **Execution time:** a speed-up of at least 100 times for n ≥ 3,200, in spite of the cost of sorting, because sorting n numbers takes far fewer operations than n(n − 1)/2 distance calculations.
- **Where the hypothesis should fail:** for drones that all share one x-coordinate, a sweep along x only should show no improvement (about n(n−1)/2 distance calculations), while the axis-choosing version should fix it. For the plus-shaped formation I predict a sweep along either axis still examines about $n^2/8$ pairs, i.e. only a four-fold reduction.

► E. Implement and validate the redesign (2 marks)

The prototype was implemented in Python (Appendix A) with counters for distance calculations and gap checks. The redesigned solution was compared with the original on the same drone positions used in Q4 (the same seeds and sizes).

n	Original: distance calculations	Redesign: distance calculations	Reduction factor	Original time (s)	Redesign time (s) incl. sort	Speed-up	Same answer?
100	4,950	149	33×	0.0005	0.00004	12×	Yes
200	19,900	415	48×	0.0019	0.00011	17×	Yes
400	79,800	852	94×	0.0078	0.00023	34×	Yes
800	319,600	1,684	190×	0.0328	0.00054	61×	Yes
1,600	1,279,200	2,460	520×	0.1303	0.00094	139×	Yes
3,200	5,118,400	5,762	888×	0.5897	0.00217	272×	Yes
6,400	20,476,800	13,759	1,488×	2.5925	0.00478	542×	Yes
12,800	81,913,600	21,327	3,841×	10.1717	0.00929	1,095×	Yes

Table 15: Original solution against redesigned solution, uniformly random drones. (Answers were compared with brute force up to n = 3,200 and the two distance values agreed in every case; for n = 6,400 and 12,800 the brute-force result of Q4 gave the same minimum distance, 1.7613 and 0.5147, as the sweep.)

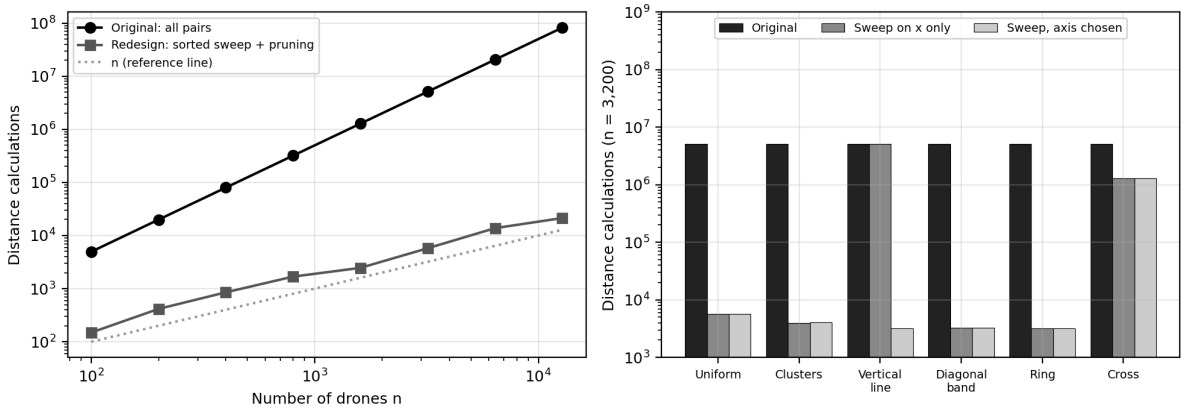


Figure 9: Left – distance calculations, original against redesign. Right – distance calculations for six data conditions at n = 3,200 (logarithmic axis).

Data condition (n = 3,200)	Original	Sweep on x only	Sweep with axis chosen	Reduction (axis chosen)	Same answer?
Uniform random	5,118,400	5,762	5,762	888×	Yes
Four clusters	5,118,400	3,985	4,050	1,264×	Yes
Vertical line (x constant)	5,118,400	5,121,599	3,199	1,600×	Yes
Near-diagonal band	5,118,400	3,259	3,286	1,558×	Yes
Points on a ring	5,118,400	3,202	3,202	1,599×	Yes
Cross (plus sign)	5,118,400	1,282,399	1,282,399	4×	Yes

Table 16: Distance calculations under different data conditions.

n	Distance calculations	Gap checks	Time (s)	Pairs a brute-force run would need
25,000	29,386	29,386	0.021	312,487,500
100,000	175,208	175,207	0.140	4,999,950,000
400,000	649,929	649,929	0.628	79,999,800,000
1,000,000	1,905,726	1,905,725	1.593	499,999,500,000

Table 17: Redesign only, for fleet sizes that brute force cannot handle. At n = 1,000,000 brute force would need about 5×10^{11} distance calculations; at the measured 0.12 µs per pair that is about 17 hours.

Validation of correctness. In addition to the comparisons above, the redesign and the brute-force solution were compared on 400 random small instances with many duplicate and tied positions (coordinates restricted to 0–20, up to 60 points); they agreed in **400 of 400** cases.

► F. Defend the final design (2 marks)

Was the prediction supported? For uniformly random drones, yes. The redesign used 5,762 distance calculations at n = 3,200, which is 1.80n and inside the predicted range of n to 6n (the point estimate 2n was slightly high). The growth was close to linear: from n = 3,200 to 12,800 (four times larger) the count rose from 5,762 to 21,327, a factor of 3.7, whereas brute force rose by 16. The predicted failure on the vertical line was also confirmed: sweeping on x alone needed 5,121,599 distance calculations, slightly more than brute force, because the seeding loop adds n − 1 calculations to an unchanged all-pairs scan, and choosing the axis restored 3,199. The plus-shaped formation behaved as predicted: 1,282,399 calculations, a reduction of only 4.0 times.

Magnitude of the improvement. Distance calculations fell by a factor of 888 at n = 3,200 and by 3,841 at n = 12,800, and the measured time (including the sort) fell by 272 and 1,095 times. Against the objectives: the correctness constraint was met in every test; the primary objective (at most 20n) was met with 1.5n to 2.2n for random data (and 1.3n for the four-cluster data); the growth objective was met (a factor of 1.5 to 2.4 per doubling for $n \geq 1{,}600$, against 4); and the operating objective was greatly exceeded, because the redesign handled 1,000,000 drones in 1.6 s and 400,000 in 0.5 s, so the 1-second limit moved from about 4,000 drones to roughly 600,000, about 150 times larger. The robustness objective was met in the sense that the failing cases were found and explained.

New costs and trade-offs. Sorting is $\Theta(n \log n)$ and becomes the dominant cost for large n (the distance calculations are only about 1.9 million at n = 1,000,000, but sorting a million points takes a large share of the 1.6 seconds); an extra copy of the data needs $\Theta(n)$ memory; and the running time now depends on the geometry of the input instead of only on n. The latter is the serious cost: the worst case is still $\Theta(n^2)$.

When the original solution may still be preferable. For a very small number of drones (a few dozen), the brute-force solution costs microseconds, has no sort or extra memory, and has an obvious proof of correctness; the gain from the redesign is then not worth the added complexity. The original is also the right choice if the system must compute *all* pair distances anyway (for example a full collision-risk matrix), or for use as an independent checker of a faster implementation, as it was here.

Does the redesign sufficiently address the limitation? It addresses the diagnosed limitation, namely that effort is spent on pairs that cannot be the answer, for all the realistic fleet patterns tested (uniform, clustered, diagonal, ring and aligned formations). It does not provide a worst-case guarantee, because formations such as the plus-shaped one still require about a quarter of all pairs. If the operator needs a guaranteed response time irrespective of formation, the next step is a design that bounds the number of points that can share a search region, for example partitioning the plane into cells whose size is derived from δ. That step is not implemented here, as the assignment does not require it. My final engineering conclusion is to deploy the sorted sweep with axis selection for fleets of any size that appear in operation, to keep the brute-force routine as a test oracle, and to monitor the distance-calculation counter: if it ever exceeds, say, 50n, the input has an unusual formation and the system should fall back to the cell-based design.

Overall Conclusion

This assignment studied how much work brute-force and exhaustive-search algorithms really do, and whether that work is acceptable. The table summarises the evidence-based decision for every question.

Question	Main evidence	Decision
Q1 Sorting	Selection: 499,500 comparisons whatever the data; Insertion: 25,009 on 95%-sorted data at n = 1,000 against about 499,000 for the others	Use Insertion Sort for the fulfilment centre (it is stable and its cost follows the disorder)
Q2 Sequential search	Cost exactly p comparisons for position p and n for a miss; limit $n \cdot q \approx 0.10/(0.6c)$	Sequential search for one hangar; hash index by Tool ID for 20 facilities
Q3 String matching	9,991 comparisons against 99,910 for the same n and m; 1.08 comparisons per shift on log text	Brute force is acceptable for ordinary logs; test worst-case and repetitive input
Q4 Closest pair	$n(n-1)/2$ pairs exactly; 1-second limit at about 4,000 drones	Brute force is a baseline only; redesign needed for larger fleets (Q7)
Q5 Convex hull	Side tests grow as n^3 (factor about 7–8 per doubling); all outputs verified geometrically and independently	Suitable for tens to a few hundred stations
Q6 Exhaustive search	2^n candidates exactly; time doubles per package; 6.6 s at 25 packages and 13.0 s at 26	Deploy for at most 24 packages (limit 25) with a 10-second response limit
Q7 Redesign of Q4	Distance calculations reduced 888 times at n = 3,200 and 3,841 times at n = 12,800; identical answers; plus-shaped input only 4 times fewer	Deploy the sorted sweep with axis choice; keep brute force as an oracle; monitor the counter

Three lessons run through all seven questions. First, the input size alone does not determine the work: Q1 and Q3 showed differences of 20 times and 10 times for inputs of the same size. Second, brute force is not wrong; it is a correct, simple baseline whose limit can be measured, as Q4, Q5 and Q6 did (about 4,000 drones, about 400 points and 25 packages for the response times chosen). Third, a redesign is justified by a diagnosis: the Q7 improvement came not from naming a better algorithm but from measuring that 99.95% of the pairs could not matter, and then using ordering and a bound to avoid them.

Final recommendations

1. **Q1:** Insertion Sort for nearly sorted orders; consider sorting the new batch and merging it if batches grow.
2. **Q2:** Keep sequential search for a single hangar; for 20 facilities, introduce a hash index by Tool ID and reassess whenever $n \cdot q$ changes.
3. **Q3:** Keep brute-force matching for typical logs, but benchmark repetitive and adversarial inputs, since the cost can be up to m times the best case.
4. **Q4 and Q7:** Replace the all-pairs scan by the sorted sweep with axis selection, retain brute force for validation, and add a counter that detects unusual formations.
5. **Q5:** Use the brute-force hull for small station sets and as a reference oracle; do not use it for large point sets.
6. **Q6:** Limit the exhaustive search to 24 packages for a 10-second response and refuse or switch method beyond that.

Limitations of this study

- Timing was measured on one machine, in Python, on a single core. The operation counts are exact and machine-independent; the times and the boundaries derived from them (4,000 drones, 25 packages, the CPU-load boundary of Q2) will change with the platform, although the growth rates will not.
- The workload figures in Q2 (500 records per hangar, 0.5 requests per second) and the response-time limits in Q4 and Q6 are my own modelling assumptions, stated where used. The assignment does not supply them.
- Random data were generated with a single fixed seed per experiment. Results for random data could differ by a few percent with another seed. The formula-based predictions, which do not depend on the seed, were therefore stated alongside the observations.
- Q7 tests a heuristic-style redesign that is exact in its answer but not guaranteed in its running time; the plus-shaped formation shows the worst case that remains.
- Larger values of n for brute force in Q4 and Q5 were extrapolated, not measured, beyond 12,800 and 400 respectively.

Mark-by-mark evidence map

Question	Marks	Evidence presented in this report
Q1	6	Predictions with reasons, seven designed data sets (two contrasting, one edge case), three implementations, counts, comparison, recommendation
Q2	4	Position/size/unsuccessful experiments, comparison counts, operating boundary, re-evaluation for 20 facilities
Q3	6	Three-plus test conditions with predictions before execution, character-comparison counts, explanation
Q4	6	Pair-count derivation, implementation, eight input sizes, acceptability criterion, origin of the inefficiency
Q5	7	Derived decision rule and pseudocode, three test configurations, visualisation, growth analysis, correctness argument
Q6	8	Bit-mask design, hand-verified instance, 2^n prediction, ten sizes, explanation and deployment boundary
Q7	13	Diagnosis (A), objective (B), redesign with pseudocode (C), prediction (D), validation (E), defence (F)

References

- Levitin, A., *Introduction to the Design and Analysis of Algorithms*, Pearson (chapters on the analysis framework and on brute force and exhaustive search: sorting, sequential search, string matching, closest pair, convex hull, knapsack).
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C., *Introduction to Algorithms*, MIT Press (chapters on growth of functions, insertion sort, convex hull and closest pair).
- Course material, CSA06 – Design and Analysis of Algorithms, SIMATS Engineering, Assignment 2 (CO2) question paper.

Appendix A – Source Code

All experiments were produced by the following Python programs (standard library only). Each program writes its raw results to a JSON file (Appendix B), and the same files are supplied with this report. Long lines have been wrapped for printing.

q1_sorting.py – Q1 – Selection, Bubble and Insertion Sort with counters

```
"""Q1 - Selection, Bubble and Insertion Sort under different data conditions.
Each routine returns (sorted_list, comparisons, swaps_or_moves)."""
import random, time, json

def selection_sort(a):
    a = a[:]; n = len(a); comp = swaps = 0
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            comp += 1
            if a[j] < a[m]:
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]; swaps += 1
    return a, comp, swaps

def bubble_sort(a):
    """Bubble sort with the usual 'no swap in a pass -> stop' flag."""
    a = a[:]; n = len(a); comp = swaps = 0
    for p in range(n - 1):
        swapped = False
        for j in range(n - 1 - p):
            comp += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]; swaps += 1; swapped = True
        if not swapped:
            break
    return a, comp, swaps

def insertion_sort(a):
    """Counts one comparison per key test and one 'move' per element shifted."""
    a = a[:]; n = len(a); comp = moves = 0
    for i in range(1, n):
        v = a[i]; j = i - 1
        while j >= 0:
            comp += 1
            if a[j] > v:
                a[j + 1] = a[j]; moves += 1; j -= 1
            else:
                break
        a[j + 1] = v
    return a, comp, moves

ALGOS = {"Selection": selection_sort, "Bubble": bubble_sort, "Insertion": insertion_sort}

def datasets(n, seed=7):
    r = random.Random(seed)
    rnd = [r.randint(1, 10 * n) for _ in range(n)]
    srt = sorted(rnd)
    rev = srt[::-1]
    dup = [r.choice([1, 2, 3]) for _ in range(n)]            # edge case: only 3 distinct scores
    eq = [5] * n                                               # edge case: all equal
    k = n // 20                                                # 5 % new orders appended
    near = sorted(r.randint(1, 10 * n) for _ in range(n - k)) + [r.randint(1, 10 * n) for _ in
        range(k)]
    # 95 % in order but the new batch is scattered (inserted at random positions)
    base = sorted(r.randint(1, 10 * n) for _ in range(n - k))
    scat = base[:]
    for _ in range(k):
        scat.insert(r.randrange(len(scat) + 1), r.randint(1, 10 * n))
    return {"Sorted": srt, "Reversed": rev, "Random": rnd, "Nearly sorted (5% appended)": near,
            "Nearly sorted (5% scattered)": scat, "Edge: 3 distinct values": dup, "Edge: all equal":
                eq}

def timeit(f, data, reps=3):
    best = 1e9
    for _ in range(reps):
        t = time.perf_counter(); f(data); best = min(best, time.perf_counter() - t)
    return best

if __name__ == "__main__":
    out = {}
```

```
for n in (1000, 2000):
    out[n] = {}
    for name, data in datasets(n).items():
        out[n][name] = {}
        for an, f in ALGOS.items():
            s, c, m = f(data)
            assert s == sorted(data), (an, name)
            out[n][name][an] = {"comparisons": c, "moves": m, "time_ms": round(1000 * timeit(f,
                data), 3)}
json.dump(out, open("../data/q1.json", "w"), indent=1)
for n in out:
    print("n =", n)
    for name in out[n]:
        print(f"  {name:32s}", {a: (v['comparisons'], v['moves'], v['time_ms']) for a, v in
            out[n][name].items()})
```

q2_search.py – Q2 – Sequential search experiments and workload simulation

```
"""Q2 - Sequential search on the temporary tool-issue list (aircraft maintenance)."""
import random, time, json

def make_records(n, seed=11):
    r = random.Random(seed)
    ids = r.sample(range(1, 10 * n + 1000), n)          # unique tool numbers
    return [(f"T-{t}", f"TECH-{r.randint(1, 60)}", f"{r.randint(6, 21):02d}:{r.randint(0, 59):02d}",
        f"BAY-{r.randint(1, 12)}")
            for t in ids]

def seq_search(records, tool_id):
    """Returns (index or -1, comparisons)."""
    comps = 0
    for i, rec in enumerate(records):
        comps += 1
        if rec[0] == tool_id:
            return i, comps
    return -1, comps

def ns_per_comparison(records, reps=5):
    """Time of an unsuccessful scan divided by its comparison count (measured cost c)."""
    best = 1e9
    for _ in range(reps):
        t = time.perf_counter(); seq_search(records, "T-none"); best = min(best, time.perf_counter()
            - t)
    return best / len(records) * 1e9

def hash_time_ns(index, key, reps=200000):
    t = time.perf_counter()
    for _ in range(reps):
        index.get(key)
    return (time.perf_counter() - t) / reps * 1e9

def simulate(N, ops=4000, seed=5):
    """Steady-state workload: 50% 'who has it?' lookups, 25% returns (search + delete), 25% issues
        (append)."""
    r = random.Random(seed); recs = make_records(N, seed); nxt = 10 ** 7
    total = {"lookup": [0, 0], "return": [0, 0]}      # [comparisons, count]
    for _ in range(ops):
        u = r.random()
        if u < 0.5:
            tgt = recs[r.randrange(len(recs))][0] if r.random() < 0.8 else "T-missing"
            _, c = seq_search(recs, tgt); total["lookup"][0] += c; total["lookup"][1] += 1
        elif u < 0.75:
            tgt = recs[r.randrange(len(recs))][0]
            i, c = seq_search(recs, tgt); total["return"][0] += c; total["return"][1] += 1
            recs.pop(i)
            recs.append((f"T-{nxt}", "TECH-1", "08:00", "BAY-1")); nxt += 1   # keep size steady
                (re-issue)
        else:
            pass
    return {k: round(v[0] / v[1], 1) for k, v in total.items()}

if __name__ == "__main__":
    out = {"sizes": {}, "sim": {}}
    for n in (10, 100, 1000, 10000, 100000):
        recs = make_records(n)
        ids = [r[0] for r in recs]
        pos = {"first": 0, "25%": n // 4, "50%": n // 2, "75%": (3 * n) // 4, "last": n - 1}
        row = {k: seq_search(recs, ids[p])[1] for k, p in pos.items()}
        step = max(1, n // 1000)                          # exact average over every position
            (sampled evenly for huge n)
        sample = range(0, n, step)
        row["average (successful)"] = round(sum(seq_search(recs, ids[p])[1] for p in sample) /
            len(sample), 1)
        row["unsuccessful"] = seq_search(recs, "T-none")[1]
        row["ns_per_comp"] = round(ns_per_comparison(recs), 1)
        idx = {r[0]: r for r in recs}
        row["hash_probes"] = 1
        row["hash_ns"] = round(hash_time_ns(idx, ids[n // 2]), 1)

        out["sizes"][n] = row
    for N in (500, 10000):
        out["sim"][N] = simulate(N)
    json.dump(out, open("../data/q2.json", "w"), indent=1)
    for n, row in out["sizes"].items(): print(n, row)
    print(out["sim"])
```

q3_string.py – Q3 – Brute-force string matching with character-comparison counter

```
"""Q3 - Brute-force string matching: counting character comparisons."""
import random, json

def brute_force(text, pat):
    """Returns (list of match positions, character comparisons, shifts tried)."""
    n, m = len(text), len(pat); comps = 0; hits = []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comps += 1
            if text[i + j] != pat[j]:
                break
            j += 1
        if j == m:
            hits.append(i)
    return hits, comps, n - m + 1

def synthetic_log(n_chars, seed=3):
    r = random.Random(seed)
    users = ["root", "admin", "guest", "deploy", "svc"]
    msgs = ["Accepted password for {u} from 10.0.{a}.{b} port {p}", "Failed password for {u} from
        10.0.{a}.{b} port {p}",
            "Connection closed by 10.0.{a}.{b}", "session opened for user {u}", "CRON job finished
                in {p} ms"]
    out = []; size = 0
    while size < n_chars:
        line = f"{r.randint(0,23):02d}:{r.randint(0,59):02d}:{r.randint(0,59):02d}
            srv{r.randint(1,9)} " + \
               r.choice(msgs).format(u=r.choice(users), a=r.randint(0, 9), b=r.randint(0, 255),
                   p=r.randint(1000, 65000)) + "\n"
        out.append(line); size += len(line)
    return "".join(out)[:n_chars]

def tests(n=10000, m=10):
    log = synthetic_log(n)
    T1 = ("B" * n, "A" + "B" * (m - 1))                                  # first character never
        matches
    T2 = ("A" * n, "A" * (m - 1) + "B")                                  # engineered worst case
    T3 = (log, "Failed password for root")                               # realistic log text
    r = random.Random(9)
    T4 = ("".join(r.choice("01") for _ in range(n)), "1" * (m - 1) + "0") # binary alphabet, random
        text
    T5 = (log, "Failed password for root"[::-1])                         # pattern absent, rare
        first char
    return {"T1 best-case (first char mismatches)": T1, "T2 engineered worst case": T2,
            "T3 realistic log, 24-char signature": T3, "T4 random binary text": T4}

if __name__ == "__main__":
    out = {"tests": {}, "scaling": []}
    n = 10000
    for name, (t, p) in tests(n).items():
        hits, c, s = brute_force(t, p)
        out["tests"][name] = {"n": len(t), "m": len(p), "shifts": s, "comparisons": c, "matches":
            len(hits), "ratio_to_shifts": round(c / s, 3)}
    # scaling of the worst case with pattern length m (text length fixed)
    for m in (2, 5, 10, 20, 50, 100):
        t, p = "A" * n, "A" * (m - 1) + "B"
        _, c, s = brute_force(t, p)
        _, c1, _ = brute_force("B" * n, "A" + "B" * (m - 1))
        out["scaling"].append({"m": m, "worst": c, "best": c1, "predicted_worst": m * (n - m + 1),
            "predicted_best": n - m + 1})
    # scaling with text length n (m = 10)
    out["scaling_n"] = []
    for n2 in (1000, 2000, 4000, 8000, 16000):
        _, c, _ = brute_force("A" * n2, "A" * 9 + "B")
        out["scaling_n"].append({"n": n2, "worst": c, "predicted": 10 * (n2 - 9)})
    json.dump(out, open("../data/q3.json", "w"), indent=1)
    for k, v in out["tests"].items(): print(k, v)
    print(out["scaling"]); print(out["scaling_n"])
```

q4_closest.py – Q4 – Brute-force closest pair

```
"""Q4 - Brute-force closest pair of drones."""
import random, time, json, math

def closest_pair_bf(P):
    """Examine every unordered pair (i<j) exactly once.
    Squared distance is used so that no square root is needed for comparison.
    Returns (best_d2, (i, j), pairs_examined, distance_calculations)."""
    n = len(P); best = float("inf"); arg = None; pairs = dists = 0
    for i in range(n - 1):
        xi, yi = P[i]
        for j in range(i + 1, n):
            pairs += 1
            dx = xi - P[j][0]; dy = yi - P[j][1]
            d2 = dx * dx + dy * dy; dists += 1
            if d2 < best:
                best = d2; arg = (i, j)
    return best, arg, pairs, dists

def random_points(n, seed=1, side=10000.0):
    r = random.Random(seed)
    return [(r.uniform(0, side), r.uniform(0, side)) for _ in range(n)]

if __name__ == "__main__":
    out = {"rows": []}
    for n in (100, 200, 400, 800, 1600, 3200, 6400, 12800):
        P = random_points(n)
        el = 1e9
        for _ in range(1 if n >= 6400 else 3):          # best of 3 for small sizes (reduces timing
            noise)
            t = time.perf_counter(); best, arg, pairs, dists = closest_pair_bf(P); el = min(el,
                time.perf_counter() - t)
        out["rows"].append({"n": n, "pairs": pairs, "formula": n * (n - 1) // 2, "dists": dists,
            "time_s": round(el, 4),
                            "min_dist": round(math.sqrt(best), 4)})
        print(out["rows"][-1])
    json.dump(out, open("../data/q4.json", "w"), indent=1)
```

q5_hull.py – Q5 – Brute-force convex hull, tests and growth experiment

```
"""Q5 - Brute-force convex hull derived from the 'all other points on one side' principle."""
import random, time, json, math

def side(p, q, r):
    """Signed value of the line through p,q evaluated at r:
    a*x + b*y - c with a = qy-py, b = px-qx, c = px*qy - py*qx.
    >0 : r on one side, <0 : other side, 0 : r on the line (collinear)."""
    a = q[1] - p[1]; b = p[0] - q[0]; c = p[0] * q[1] - p[1] * q[0]
    return a * r[0] + b * r[1] - c

def on_segment(p, q, r):
    return min(p[0], q[0]) <= r[0] <= max(p[0], q[0]) and min(p[1], q[1]) <= r[1] <= max(p[1], q[1])

def convex_hull_bf(points):
    """Returns (hull vertices in counter-clockwise order, edges, pairs examined, side tests)."""
    pts = sorted(set(points)); n = len(pts)
    if n < 3:
        return pts, [], 0, 0
    pairs = tests = 0; edges = []
    for i in range(n - 1):
        for j in range(i + 1, n):
            pairs += 1
            pos = neg = False; ok = True
            for k in range(n):
                if k == i or k == j:
                    continue
                tests += 1
                s = side(pts[i], pts[j], pts[k])
                if s > 0: pos = True
                elif s < 0: neg = True
                elif not on_segment(pts[i], pts[j], pts[k]):
                    ok = False; break                    # collinear but outside the segment: a
                        longer edge exists
                if pos and neg:
                    ok = False; break                    # points on both sides: not a boundary edge
            if ok:
                edges.append((i, j, pos, neg))
    if not edges:
        return pts, [], pairs, tests
    # all points collinear: hull degenerates to the two extreme points
    if all(not e[2] and not e[3] for e in edges):
        return [pts[edges[0][0]], pts[edges[0][1]]], edges, pairs, tests
    # orient each edge so that every other point lies to its left (gives a counter-clockwise boundary)
    nxt = {}
    for i, j, pos, neg in edges:
        if pos and not neg:   # all others have positive value (they lie to the right of i->j) ->
            reverse the edge
            i, j = j, i
        nxt[i] = j
    start = min(nxt); hull = [pts[start]]; cur = nxt[start]
    while cur != start:
        hull.append(pts[cur]); cur = nxt[cur]
    return hull, edges, pairs, tests

def monotone_chain(points):
    """Independent reference (Andrew) used ONLY to cross-check the brute-force result."""
    P = sorted(set(points))
    if len(P) < 3: return P
    cr = lambda o, a, b: (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])
    lo = []
    for p in P:
        while len(lo) >= 2 and cr(lo[-2], lo[-1], p) <= 0: lo.pop()
        lo.append(p)
    up = []
    for p in reversed(P):
        while len(up) >= 2 and cr(up[-2], up[-1], p) <= 0: up.pop()
        up.append(p)
    return lo[:-1] + up[:-1]

def same_hull(h1, h2):

    return set(h1) == set(h2) and len(h1) == len(h2)

def test_sets():
    r = random.Random(4)
    A = [(r.randint(0, 100), r.randint(0, 100)) for _ in range(30)]                      # random
        boundary + interior
    G = [(x, y) for x in range(0, 5) for y in range(0, 5)]                                  # 5x5
        grid: many collinear points
    G += [(2, 2), (0, 0)]                                                                  #
        duplicate points
    C = [(round(50 + 40 * math.cos(2 * math.pi * k / 12)), round(50 + 40 * math.sin(2 * math.pi * k
        / 12))) for k in range(12)]
```

```
    C += [(50, 50), (45, 52), (55, 48), (50, 60), (38, 50)]                                  # all
        on-circle + interior
    return {"Test 1: random points": A, "Test 2: 5x5 grid with duplicates (collinear stress)": G,
        "Test 3: circle + interior points": C}

def random_pts(n, seed=2):
    r = random.Random(seed)
    return [(r.uniform(0, 1000), r.uniform(0, 1000)) for _ in range(n)]

def circle_pts(n):
    return [(500 + 400 * math.cos(2 * math.pi * k / n), 500 + 400 * math.sin(2 * math.pi * k / n))
        for k in range(n)]

if __name__ == "__main__":
    out = {"tests": {}, "growth": []}
    for name, pts in test_sets().items():
        h, e, p, t = convex_hull_bf(pts)
        ref = monotone_chain(pts)
        out["tests"][name] = {"n_input": len(pts), "n_distinct": len(set(pts)), "hull": h, "edges":
            len(e), "pairs": p, "tests": t, "matches_reference": same_hull(h, ref)}
        print(name, out["tests"][name])
    for n in (25, 50, 100, 200, 400):
        row = {"n": n}
        for lab, pts in (("random", random_pts(n)), ("circle", circle_pts(n))):
            t0 = time.perf_counter(); h, e, p, t = convex_hull_bf(pts); el = time.perf_counter() - t0
            assert same_hull(h, monotone_chain(pts)), (lab, n)
            row[lab] = {"hull": len(h), "pairs": p, "tests": t, "time_s": round(el, 4),
                "worst_tests": p * (n - 2)}
        out["growth"].append(row); print(row)
    json.dump(out, open("../data/q5.json", "w"), indent=1)
```

q6_exhaustive.py – Q6 – Exhaustive search (plain and Gray-code versions)

```
"""Q6 - Exhaustive search for the helicopter payload problem (maximise value, weight <= limit)."""
import random, time, json

def exhaustive_plain(w, v, W):
    """Candidate = bit mask (bit k set <=> package k loaded). All 2^n masks are generated,
    weight and value are recomputed from scratch for each mask (clearest version)."""
    n = len(w); best_v = -1; best_mask = 0; feasible = 0
    for mask in range(1 << n):
        tw = tv = 0
        for k in range(n):
            if mask >> k & 1:
                tw += w[k]; tv += v[k]
        if tw <= W:
            feasible += 1
            if tv > best_v:
                best_v, best_mask = tv, mask
    return best_v, best_mask, 1 << n, feasible

def exhaustive_gray(w, v, W):
    """Same 2^n candidates, visited in Gray-code order so that successive candidates differ by ONE
    package; weight and value are updated in O(1) per candidate. No candidate is skipped or pruned."""
    n = len(w); tw = tv = 0; mask = 0; best_v = 0; best_mask = 0; feasible = 1   # empty load is
        feasible
    for k in range(1, 1 << n):
        b = (k & -k).bit_length() - 1
        if mask >> b & 1:
            mask ^= 1 << b; tw -= w[b]; tv -= v[b]
        else:
            mask ^= 1 << b; tw += w[b]; tv += v[b]
        if tw <= W:
            feasible += 1
            if tv > best_v:
                best_v, best_mask = tv, mask
    return best_v, best_mask, 1 << n, feasible

def dp_reference(w, v, W):
    """Independent check (dynamic programming) - used only to verify the optimum value."""
    dp = [0] * (W + 1)
    for wi, vi in zip(w, v):
        for c in range(W, wi - 1, -1):
            dp[c] = max(dp[c], dp[c - wi] + vi)
    return dp[W]

def instance(n, seed=21):
    r = random.Random(seed)
    w = [r.randint(5, 40) for _ in range(n)]
    v = [r.randint(10, 100) for _ in range(n)]
    return w, v, sum(w) // 2                      # payload limit = half of the total weight

def names(mask, n):
    return [f"P{k+1}" for k in range(n) if mask >> k & 1]

if __name__ == "__main__":
    out = {}
    # ---- small hand-checkable instance (4 packages) ----
    w, v, W = [7, 3, 4, 5], [42, 12, 40, 25], 10
    rows = []
    for mask in range(16):
        tw = sum(w[k] for k in range(4) if mask >> k & 1); tv = sum(v[k] for k in range(4) if mask
            >> k & 1)
        rows.append({"mask": format(mask, "04b"), "set": names(mask, 4), "w": tw, "v": tv,
            "feasible": tw <= W})
    bv, bm, cand, feas = exhaustive_plain(w, v, W)
    out["small"] = {"w": w, "v": v, "W": W, "rows": rows, "best_v": bv, "best_set": names(bm, 4),
        "candidates": cand, "feasible": feas,
                    "dp": dp_reference(w, v, W), "gray": exhaustive_gray(w, v, W)[0]}
    # ---- random cross-validation against DP ----
    r = random.Random(1); ok = 0; trials = 300
    for _ in range(trials):
        n = r.randint(1, 14); ww = [r.randint(1, 30) for _ in range(n)]; vv = [r.randint(1, 90) for

            _ in range(n)]; WW = r.randint(1, sum(ww))
        a = exhaustive_plain(ww, vv, WW)[0]; b = exhaustive_gray(ww, vv, WW)[0]; c =
            dp_reference(ww, vv, WW)
        ok += (a == b == c)
    out["validation"] = {"trials": trials, "agree": ok}
    # ---- growth experiment ----
    out["growth"] = []
    for n in (10, 12, 14, 16, 18, 20, 22, 24):
        w, v, W = instance(n)
        t = time.perf_counter(); bv, bm, cand, feas = exhaustive_gray(w, v, W); el =
            time.perf_counter() - t
        row = {"n": n, "candidates": cand, "feasible": feas, "best_value": bv, "time_s": round(el,
```

```
            4), "dp": dp_reference(w, v, W)}
        if n <= 16:
            t = time.perf_counter(); pv = exhaustive_plain(w, v, W)[0]; row["plain_time_s"] =
                round(time.perf_counter() - t, 4); row["plain_value"] = pv
        out["growth"].append(row); print(row)
    json.dump(out, open("../data/q6.json", "w"), indent=1)
    print(out["small"]["best_v"], out["small"]["best_set"], out["small"]["dp"], out["validation"])
```

q7_redesign.py – Q7 – Sorted-sweep redesign of the closest-pair solution and its validation

```
"""Q7 - Redesign of the brute-force closest-pair solution (Q4): sorted sweep with distance-bound
    pruning."""
import random, time, json, math
from q4_closest import closest_pair_bf, random_points

def closest_pair_sweep(P, choose_axis=True):
    """1. Pre-processing: sort the points along one axis (the axis with the larger spread if
        choose_axis).
    2. Seed the best distance with the best *adjacent* pair in sorted order (n-1 distance
        calculations).
    3. For each point i, scan forward only while the axis gap to point j is still smaller than the
       best distance found so far; the first point whose gap is too large ends the scan for i,
       because every later point is even farther along the axis.
    Returns (best_d2, distance_calcs, gap_checks)."""
    n = len(P)
    ax = 0
    if choose_axis:
        xs = [p[0] for p in P]; ys = [p[1] for p in P]
        ax = 0 if (max(xs) - min(xs)) >= (max(ys) - min(ys)) else 1
    Q = sorted(P, key=lambda p: p[ax])
    best = float("inf"); dists = 0; checks = 0
    for i in range(n - 1):                                  # seed
        dx = Q[i][0] - Q[i + 1][0]; dy = Q[i][1] - Q[i + 1][1]
        d2 = dx * dx + dy * dy; dists += 1
        if d2 < best: best = d2
    for i in range(n - 1):
        ci = Q[i][ax]; xi, yi = Q[i]
        for j in range(i + 1, n):
            g = Q[j][ax] - ci; checks += 1
            if g * g >= best:
                break
            dx = xi - Q[j][0]; dy = yi - Q[j][1]
            d2 = dx * dx + dy * dy; dists += 1
            if d2 < best: best = d2
    return best, dists, checks

def useful_fraction(P, d_min):
    """Diagnosis helper: share of all pairs whose x-gap is smaller than the final minimum distance,
    i.e. the only pairs that could possibly beat it."""
    xs = sorted(p[0] for p in P); n = len(xs); cnt = 0; j = 0
    for i in range(n):
        while j < n and xs[j] - xs[i] < d_min: j += 1
        cnt += j - i - 1
    return cnt, n * (n - 1) // 2

def datasets(n):
    r = random.Random(17)
    uni = random_points(n, seed=1)
    clu = [(r.gauss(cx, 150), r.gauss(cy, 150)) for cx, cy in [(2000, 2000), (7000, 3000), (4000,
        8000), (8500, 8500)] for _ in range(n // 4)]
    vert = [(5000.0, r.uniform(0, 10000)) for _ in range(n)]
    diag = [(t, t + r.uniform(-5, 5)) for t in (r.uniform(0, 10000) for _ in range(n))]
    circ = [(5000 + 4000 * math.cos(a), 5000 + 4000 * math.sin(a)) for a in (r.uniform(0, 2 *
        math.pi) for _ in range(n))]
    cross = [(5000.0, r.uniform(0, 10000)) for _ in range(n // 2)] + [(r.uniform(0, 10000), 5000.0)
        for _ in range(n - n // 2)]
    return {"Uniform random": uni, "Four clusters": clu, "Vertical line (x constant)": vert,
        "Near-diagonal band": diag, "Points on a ring": circ, "Cross (plus sign)": cross}

def timed(f, *a, reps=3):
    best = 1e9
    for _ in range(reps):
        t = time.perf_counter(); res = f(*a); best = min(best, time.perf_counter() - t)
    return res, best

if __name__ == "__main__":
    out = {"diagnosis": [], "compare": [], "datasets": [], "validation": {}, "large": []}
    # diagnosis from the ORIGINAL solution
    for n in (400, 1600, 3200):
        P = random_points(n); bd = closest_pair_bf(P)[0]; useful, total = useful_fraction(P,

            math.sqrt(bd))
        out["diagnosis"].append({"n": n, "total_pairs": total, "pairs_with_small_x_gap": useful,
```

```
        "percent": round(100 * useful / total, 4)})
# original vs redesign, uniform random, same points as Q4
for n in (100, 200, 400, 800, 1600, 3200, 6400, 12800):
    P = random_points(n)
    (b2, d1, c1), t1 = timed(closest_pair_sweep, P, False, reps=3)
    (b3, d2, c2), t2 = timed(closest_pair_sweep, P, True, reps=3)
    ref = closest_pair_bf(P)[0] if n <= 3200 else None
    out["compare"].append({"n": n, "bf_dists": n * (n - 1) // 2, "sweep_dists": d2,
        "sweep_checks": c2, "sweep_time_s": round(t2, 5),
                           "xonly_dists": d1, "correct": (ref is None or abs(ref - b3) < 1e-9)})
    print(out["compare"][-1])
# different data conditions at n = 3200
n = 3200
for name, P in datasets(n).items():
    bf = closest_pair_bf(P)
    (bx, dx_, cx), tx = timed(closest_pair_sweep, P, False, reps=1)
    (ba, da, ca), ta = timed(closest_pair_sweep, P, True, reps=1)
    row = {"name": name, "n": len(P), "bf_dists": bf[3], "xonly_dists": dx_, "axis_dists": da,
        "ok": abs(bf[0] - ba) < 1e-9 and abs(bf[0] - bx) < 1e-9,
           "axis_time_s": round(ta, 5)}
    out["datasets"].append(row); print(row)
# randomised validation including duplicates and ties
r = random.Random(99); ok = 0; T = 400
for _ in range(T):
    m = r.randint(2, 60); P = [(r.randint(0, 20), r.randint(0, 20)) for _ in range(m)]
    ok += abs(closest_pair_bf(P)[0] - closest_pair_sweep(P)[0]) < 1e-9
out["validation"] = {"trials": T, "agree": ok}
# redesign only, large inputs
for n in (25000, 100000, 400000, 1000000):
    P = random_points(n)
    (b, d, c), t = timed(closest_pair_sweep, P, True, reps=1)
    out["large"].append({"n": n, "dists": d, "checks": c, "time_s": round(t, 3), "bf_pairs": n *
        (n - 1) // 2}); print(out["large"][-1])
json.dump(out, open("../data/q7.json", "w"), indent=1)
print(out["diagnosis"], out["validation"])
```

Appendix B – Raw Experimental Data

The tables below contain raw data that were not shown in full in the main text. The complete outputs are stored in the files q1.json to q7.json that accompany this report.

Q1 – Sorting, n = 2,000 (counts and time)

Data set (n = 2,000)	Sel. comps	Sel. swaps	Bub. comps	Bub. swaps	Ins. comps	Ins. moves
Sorted	1,999,000	0	1,999	0	1,999	0
Reversed	1,999,000	1,047	1,999,000	1,998,899	1,998,998	1,998,899
Random	1,999,000	1,993	1,997,965	1,008,794	1,010,785	1,008,794
Near-sorted (5% appended)	1,999,000	1,961	1,997,569	92,730	94,729	92,730
Near-sorted (5% scattered)	1,999,000	1,829	1,984,804	59,607	61,606	59,607
3 distinct values	1,999,000	1,342	1,763,359	661,586	663,584	661,586
all equal	1,999,000	0	1,999	0	1,999	0

Execution times for these runs (milliseconds) are stored in q1.json.

Q2 – Dynamic workload simulation (4,000 operations)

Active records N	Average comparisons per lookup	Average comparisons per return	Predicted lookup (0.6N)	Predicted return ((N+1)/2)
500	296.7	253.3	300.0	250.5
10,000	6,017.7	5,104.5	6,000.0	5,000.5

Q2 – Measured cost per comparison and hash lookup

n	ns per comparison (scan)	ns per hash lookup
10	56.6	29.1
100	36.6	31.0
1,000	45.8	27.7
10,000	43.6	31.2
100,000	44.4	29.4

Q6 – Exhaustive search, plain version timings and DP check

n	Candidates	Feasible	Best value (Gray)	Best value (DP check)	Gray-code time (s)	Plain time (s)
10	1,024	514	322	322	0.0002	0.0006
12	4,096	2,059	407	407	0.0007	0.0028
14	16,384	8,246	511	511	0.0029	0.0132
16	65,536	32,988	637	637	0.0117	0.0744
18	262,144	131,072	683	683	0.0561	not run
20	1,048,576	524,288	820	820	0.1863	not run
22	4,194,304	2,110,889	835	835	0.9125	not run
24	16,777,216	8,440,288	887	887	2.9950	not run
25	33,554,432	16,777,216	990	990	6.6283	not run
26	67,108,864	33,554,432	993	993	13.0252	not run

Q7 – Gap checks and times for the redesign (uniform random drones)

n	Brute-force pairs	Redesign distance calculations	Gap checks	x-only sweep calculations	Redesign time (s)
100	4,950	149	149	147	0.00004
200	19,900	415	413	400	0.00011
400	79,800	852	851	852	0.00023
800	319,600	1,684	1,684	1,702	0.00054
1,600	1,279,200	2,460	2,460	2,460	0.00094
3,200	5,118,400	5,762	5,762	5,762	0.00217
6,400	20,476,800	13,759	13,759	13,759	0.00478
12,800	81,913,600	21,327	21,326	21,327	0.00929